

Resumen de Validación y Verificación

Índice

Unidad 1 - Riesgos

1. ¿Qué es un riesgo?
2. ¿Qué tipo de estrategias se utilizan?
3. ¿Cómo se separan los riesgos?
4. ¿Qué es la administración de riesgos?
5. ¿Cuáles son los principios de la administración de riesgos?
6. ¿Cuáles son las etapas del proceso de gestión de riesgos?
7. ¿Qué es el plan MMMR?
8. ¿Por qué es importante la gestión de riesgos en la validación y verificación de software?

Unidad 2 - VyV

1. ¿Qué es la calidad? ¿Y la calidad de software?
2. ¿Que es un proceso eficaz, producto útil y agregar valor para el usuario?
3. ¿Cuáles son los factores de calidad que se persiguen?
4. ¿Cual es el dilema de la calidad?
5. ¿Cómo se logra la calidad?
6. ¿Qué es la verificación?
7. ¿Qué es la validación?
8. ¿Cual es el objetivo de la VyV?
9. ¿De qué depende el nivel de confianza?
10. ¿En qué momento se realiza el V y V?
11. ¿Qué tipos de análisis y comprobaciones existen dentro del V y V?
12. ¿La validación se puede probar con técnicas estáticas?
13. ¿Que tipos de pruebas existen?
14. ¿Cuál es la diferencia entre V y V y depuración?
15. ¿Cómo se planifica? ¿Para que sirve el plan de pruebas?
16. ¿Qué es una inspección? ¿Qué ventajas posee sobre las pruebas?
17. ¿Cómo es el proceso de inspección de software?
18. ¿Que es un analisis estatico automatico?
19. ¿Qué son los métodos formales?
20. ¿Qué es el desarrollo de software de sala limpia?

Unidad 3 - Revisiones

1. ¿Qué son las revisiones? ¿Para qué sirven?
2. ¿Cuál es el objetivo principal de las revisiones técnicas?
3. ¿Cuáles son las consecuencias de los defectos del software?
4. ¿Qué métricas se utilizan?
5. ¿Qué es el espectro de formalidad?

6. ¿Qué es el equipo de revisión?
7. ¿Cuáles son las categorías de revisiones?
8. ¿Qué es una revisión informal? ¿Qué es una lista de revisión?
9. ¿Qué es una revisión formal?
10. ¿Cómo son las reuniones RFT? ¿Qué es un reporte técnico formal?
11. ¿Cuáles son los lineamientos para realizar una revisión?

Unidad 4 - Pruebas de software

1. ¿Cómo es un proceso de pruebas?
2. ¿Cuáles son las dos actividades fundamentales de pruebas?
3. ¿Qué objetivos posee el proceso de pruebas?
4. ¿Cuándo una prueba tiene éxito?
5. ¿Las pruebas pueden demostrar que el software está libre de defecto?
6. ¿Cómo es el modelo general del proceso?
7. ¿Se debe probar todo?
8. ¿Qué son las pruebas del sistema?
9. ¿Qué son las pruebas de integración? ¿Cómo es el proceso de integración? ¿Qué son las pruebas de regresión?
10. ¿Qué son las pruebas de entrega?
11. ¿Cuáles son las pruebas de rendimiento?
12. ¿Qué son las pruebas de componentes o de unidad?
13. ¿Cuáles son las pruebas de interfaces?
14. ¿Cómo se diseñan los casos de prueba?
15. ¿Qué tipo de pruebas se necesitan para los casos de pruebas?
16. ¿Qué es la automatización de pruebas?
17. ¿Qué es una aproximación?
18. ¿Qué es una estrategia de pruebas de software? ¿Qué debe incorporar?
19. ¿Qué es una plantilla en este contexto? ¿Qué tipo de pruebas debe incluir una estrategia de pruebas?
20. ¿Cuáles son las actividades SQA?
21. ¿Qué circunstancias pueden afectar las pruebas de un desarrollador? ¿Qué es el GPI?
22. ¿Cómo se separan las pruebas? Explique
23. ¿Qué criterios se deben tener para completar las pruebas?
24. ¿Qué aspectos se deben tener en cuenta?
25. ¿Qué tipo de estrategia se utiliza?
26. Explique las pruebas de unidad
27. ¿Qué consideraciones se deben tener en las pruebas de unidad ?
28. Explique las pruebas de integración
29. ¿Qué es un módulo crítico? ¿Cuáles son los productos de trabajo de las pruebas de integración?
30. ¿Cuáles son las estrategias para pruebas orientadas a objetos?
31. Explique las pruebas de validación ¿Cuáles son sus criterios?

32. ¿Qué son las pruebas alfa y beta?
33. Explique las pruebas de sistemas
34. Explique la depuración y sus estrategias

Unidad 5 - Pruebas orientadas a objetos

1. ¿Qué se necesita para probar los sistemas OO?
2. ¿Qué significa probar en OO?
3. ¿Cuáles son los modelos AOO y DOO?
4. ¿Qué es una prueba de unidad en OO?
5. ¿Qué es una prueba de integración en OO?
6. ¿Qué es una prueba de validación en OO?
7. ¿Cuáles son los métodos de pruebas orientados a objetos?
8. ¿Cuáles son las implicaciones del diseño de casos de prueba OO?
9. ¿Cuáles son los métodos de pruebas aplicados en nivel unidad?
10. ¿Cómo es el diseño de pruebas interclase?
11. ¿Qué es una rebanada?
12. ¿Qué es un grafo de llamada?
13. ¿Qué es la matriz de uso mínima?
14. ¿Cómo se hace una secuencia de métodos?
15. ¿Cómo se generan los casos de prueba?
16. ¿Cómo funciona este método con clases derivadas?

Unidad 6 - Pruebas de integración de sistemas y validación

Unidad 7 - Ciclo de vida del Testing

1. ¿Cómo se separa el ciclo de vida del testing? ¿Son secuenciales?
2. ¿Qué se hace en la planificación y control de pruebas?
3. ¿Qué se hace en el análisis y diseño de pruebas?
4. ¿Qué se hace en la implementación y ejecución de las pruebas?
5. ¿Qué se hace en la evaluación de los criterios de salida y reporte de pruebas?
6. ¿Qué se hace en las actividades de cierre de prueba?

Capítulo 18 - Roger S Pressman 7ma ed (técnicas de pruebas)

1. ¿Qué fundamentos existen para las pruebas del software?
2. ¿Qué visiones existen de las pruebas?
3. ¿Qué son las pruebas de caja blanca y que tipos existen?
4. ¿Qué son las pruebas de caja negra y que tipos existen?
5. ¿Qué pruebas existen para considerar arquitectura, entornos, etc?
6. ¿Qué son los patrones de prueba?

Preguntas del final

1. ¿Por qué es importante la gestión de riesgos en la validación y verificación de software?
2. ¿Cuáles son las categorías de riesgos?
3. ¿Qué es un caso de prueba? Explique dos técnicas de cada enfoque de casos de prueba (caja blanca y caja negra)

4. ¿Qué es una inspección de software? Explique los pasos de las inspecciones de software
5. ¿Cuáles son los pasos del ciclo de vida del testing? Explique que se hace en cada paso del ciclo de vida del testing
6. Enumerar y explicar los distintos tipos de riesgos
7. ¿Por qué es importante la gestión de riesgos en VyV?
8. ¿Qué es la inspección de programas?
9. Nombre y detalle los pasos de la inspección de programas.

Preguntas que no entiendo pero le pregunte a la IA

1. ¿Que es un caso de prueba?
2. ¿Qué relación hay entre el plan de pruebas y el ciclo de vida del testing?
3. Explicame el Ciclo de vida del testing resumidamente
4. ¿Qué es una condición de prueba?
5. ¿Cuál es la diferencia entre diseñar e implementar?
6. ¿Qué significa evaluar la capacidad de las pruebas?
7. ¿Los métodos de caja blanca y caja negra son para diseñar casos de prueba?
8. ¿Qué son las inspecciones y qué diferencia existe con las revisiones?
9. ¿Las pruebas en sí validan y verifican?
10. Explique los enfoques del diseño de pruebas

Unidad 1 - Riesgos

Libros:

Cap 28 -  Roger S Pressman - Ingeniería del software _ un enfoque práctico (7...

Cap 5.4 -  Ian.Sommerville-7maEdicion-Ingenieria.de.Software.pdf

Preguntontas:

1. ¿Qué es un riesgo?

Un **riesgo** es la probabilidad de que una circunstancia adversa ocurra. Son una amenaza para el proyecto, para el software que se está desarrollando y para la organización. En sí un riesgo posee dos características: La **incertidumbre** (No se sabe si pasará) y la pérdida (Que sucederá si pasa).

2. ¿Qué tipo de estrategias se utilizan?

Las estrategias que se pueden utilizar para manejar los riesgos son:

- Estrategias **reactivas**: Estas se realizan una vez ocurre el riesgo y se ven las consecuencias para actuar según dichas consecuencias. Actúan como los bomberos, cuando ocurre un incendio, salen a apagarlo, es una misma analogía.
- Estrategias **proactivas**: Estas se realizan antes de iniciar el desarrollo, se identifican los riesgos, se analiza su probabilidad e impacto que posee, se clasifican por importancia y el equipo software realiza algún tipo de plan para gestionarlos. El objetivo de esta estrategia es evitar los riesgos pero si llegan a ocurrir deben planificar un plan de contingencia en caso de que ocurra.

3. ¿Cómo se separan los riesgos?

Para identificar el impacto y la probabilidad de que ocurran se decide separar los riesgos en diferentes tipos como:

- **Riesgos de proyecto**: Son aquellos que amenazan al plan del proyecto, esto puede generar retrasos en el calendario propuestos, que los costos aumenten, etc. Estos identifican problemas de presupuestos, calendario, personal, recursos, participantes y requisitos
- **Riesgos técnicos**: Son aquellos que amenazan la calidad y temporalidad del software que se va a producir, esto puede generar que la implementación sea difícil o imposible. Estos identifican problemas de diseño, implementación,

interfaz, verificación y mantenimiento. Otros problemas también son la ambigüedad en la especificación, la incertidumbre técnica, la obsolescencia técnica y la tecnología de “punta”. Para finalizar una falla en la estimación de la complejidad del problema.

- **Riesgos empresariales:** Estos amenazan la viabilidad del software que se va a construir y ponen en peligro el proyecto o el producto. Los principales problemas son:
 - **Riesgo del mercado.** Es construir un producto o sistema excelente que no se quiere.
 - **Riesgo estratégico.** Construir un producto que no encaje con la estrategia empresarial global de la compañía.
 - **Riesgo de venta.** Construir un producto que el equipo de ventas no sepa vender.
 - **Riesgo administrativo.** Perder el apoyo de los administradores debido a un cambio de enfoque o en el personal.
 - **Riesgo presupuestales.** Perder el apoyo presupuestal o de personal.

Otra clasificación puede ser por:

- **Riesgos conocidos:** Son aquellos que se pueden descubrir después de una evaluación cuidadosa del plan del proyecto, del entorno empresarial o técnico. (Inspecciones)
- **Riesgos predecibles:** Son aquellos que ocurrieron de otros proyectos.
- **Riesgos impredecibles:** Son difíciles de identificar.

4. ¿Qué es la administración de riesgos?

La administración de riesgos son las acciones que ayudan al equipo de software a entender y manejar la incertidumbre.

5. ¿Cuáles son los principios de la administración de riesgos?

Los principios de la administración de riesgos son:

- **Mantener una perspectiva global:** Ver los riesgos dentro del contexto de un sistema donde el riesgo es un componente y el problema empresarial que se pretende resolver.
- **Tomar una visión de previsión:** Pensar que los riesgos pueden surgir en el futuro, establecer planes de contingencia de modo que sean manejables.

- **Alentar la comunicación abierta:** Si alguien menciona un riesgo potencial, no lo ignore. Si surge de manera informal, considéralo.
- **Integrar:** Una consideración de riesgo debe integrarse en el proceso del software.
- **Enfatizar un proceso continuo:** El equipo de monitorear los riesgos identificados, deben modificar su información y agregar nuevos.
- **Desarrollar una visión de producto compartida:** Si todos los participantes poseen una misma visión del software, es probable que haya mejor identificación y valoración del riesgo
- **Alentar el trabajo en equipo:** Las cualidades de todos los participantes deben reunirse cuando se realiza una administración de riesgos.

6. ¿Cuáles son las etapas del proceso de gestión de riesgos?

Las etapas del proceso de gestión de riesgos son:

- **Identificación de riesgos:** Se intenta especificar las amenazas al plan de proyecto. Existen dos tipos de riesgos en los riesgos conocidos y predecibles:
 - Los riesgos genéricos (Ocurren en todos los proyectos software)
 - Los riesgos específicos (son únicamente de los proyectos, se necesita de una persona que conozca todo el plan de proyecto).

Una forma de identificar riesgos es crear una lista de verificación de ítem de riesgo, en la cual se identifica el riesgo y se lo clasifica en subcategorías genéricas como complejidad, impacto, definición del proceso, etc.

- **Análisis de riesgos:** En la teoría se deben asignar recursos a cada riesgos que se haya identificado pero en la práctica asignar un recurso es asignar una persona que se le paga para hacer eso por lo que se decide que riesgos se van a priorizar mediante la probabilidad de su ocurrencia y el impacto del mismo.
- **Planeación de riesgos:** Una vez que los riesgos hayan sido priorizados, según el criterio de la empresa o del gerente se deben realizar alguno de estos tipos de planes:
 - Planes de **mitigación:** Estos planes son para mitigar o disminuir la probabilidad de que el riesgo ocurra.
 - Planes de **minimización:** Estos planes son para disminuir el impacto del riesgo.

- Planes de **contingencia**: Estos planes son para seguir un procedimiento en caso de que el riesgo suceda.
- **Supervisión o monitorización de riesgos**: A medida que va pasando el tiempo se van revisando el estado de los riesgos para aumentar o disminuir su probabilidad e impacto

7. ¿Qué es el plan MMMR?

El plan MMMR (mitigación, monitoreo y manejo de riesgos) documenta todo el trabajo realizado como parte del análisis de riesgo y el gerente del proyecto lo usa como parte del plan del proyecto global. Este plan se puede realizar una vez se obtiene la especificación de requerimientos y una vez iniciado el desarrollo se pone en marcha el monitoreo y mitigación de los riesgos.

8. ¿Por qué es importante la gestión de riesgos en la validación y verificación de software?

La gestión de riesgos es importante en la verificación y validación ya que permite identificar, evaluar y mitigar posibles problemas antes de que se conviertan en fallos graves, asegurando así un desarrollo más eficiente y un producto final de mayor calidad y confiabilidad.

Unidad 2 - VyV

Libros:

Cap 8 - ■ Roger S Pressman-Ingenieria.del.Software-5ta edicion.pdf

Cap 14 - ■ Roger S Pressman - Ingeniería del software _ un enfoque práctico (7...

Cap 22 - ■ Ian.Sommerville-7maEdicion-Ingenieria.de.Software.pdf

Preguntontas:

1. ¿Qué es la calidad? ¿Y la calidad de software?

La calidad es el conjunto de características tangibles o intangibles que es subjetivo al área en el que se encuentre la persona, es decir, un programador ve otro tipo de calidad que un cliente y así.

La calidad de software se define como un *proceso eficaz de software que se aplica de manera que crea un producto útil y que proporciona valor medible a aquellos que lo producen y lo utilizan.*

2. ¿Que es un proceso eficaz, producto útil y agregar valor para el usuario?

Un **proceso eficaz** es la infraestructura necesaria para elaborar un software como la administración, las buenas prácticas, las actividades sombrilla, etc, esto puede ser aquellas actividades de administración para la verificación y validación que evitan que el proyecto caiga en un caos.

Un **producto útil** entrega contenido, funciones y características que el usuario final desea. También, se entrega libre de errores, de forma confiable y que cumple con los requerimientos del software.

Al **agregar valor para el productor y para el usuario** hace referencia a que al producir software de alta calidad trae beneficios tanto a la organización como a los clientes. Al desarrollar un software de alta calidad, se requiere menos esfuerzo en el mantenimiento, menos errores que corregir, poca asistencia al cliente, etc

3. ¿Cuáles son los factores de calidad que se persiguen?

Los factores propuestos por el estándar ISO 9126 para los productos de software son:

- **Funcionalidad:** Grado en el que el software satisface las necesidades planteadas según las establecen los atributos siguientes:
 - Adaptabilidad.
 - Exactitud.
 - Interoperabilidad.
 - Cumplimiento.
 - Seguridad.
- **Confiabilidad:** Cantidad de tiempo que el software se encuentra disponible para su uso. Según lo indican los siguientes atributos:
 - Madurez.
 - Tolerancia a fallas.
 - Recuperación.
- **Usabilidad:** Grado en el que el software es fácil de usar, según lo indican los siguientes subatributos:
 - Entendible.
 - Aprendible.
 - Operable.
- **Eficiencia:** Grado en el que el software emplea óptimamente los recursos del sistema, según lo indican los subatributos siguientes:
 - Comportamiento del tiempo y de los recursos.
- **Facilidad de recibir mantenimiento:** Facilidad con la que pueden efectuarse reparaciones al software, según lo indican los atributos que siguen:
 - Analizable.
 - Cambiable.
 - Estable.
 - Susceptible de someterse a pruebas.
- **Portabilidad:** Facilidad con la que el software puede llevarse de un ambiente a otro según lo indican los siguientes atributos:
 - Adaptable.
 - Instalable.
 - Conformidad.
 - Sustituible.

4. ¿Cual es el dilema de la calidad?

El dilema de la calidad hace referencia que debe haber un equilibrio entre la calidad que se obtiene con la que se desea ya que si se dedica mucho esfuerzo en la calidad se pueden agotar los recursos o no se puede utilizar en tiempo del mercado por lo que se perdería la venta. En sí, hay que entregar un producto lo suficientemente bueno que hace referencia a que las funcionalidades que necesitan los clientes funcionen correctamente y las que desean se vayan añadiendo a medida que se van lanzando actualizaciones.

5. ¿Cómo se logra la calidad?

La calidad se asegura aportando calidad al proceso, es decir, aplicando las actividades de un ingeniero de software como el control de calidad, la administración de proyectos y el aseguramiento de calidad

6. ¿Qué es la verificación?

La **verificación** consiste en comprobar que el software concuerde con su especificación de requerimientos ya sea funcionales como no funcionales.

Este responde a la pregunta **¿Estamos construyendo el producto correctamente?**

7. ¿Qué es la validación?

La **validación** asegura que el software cumpla con la expectativa del cliente, ya que no muchas veces lo que quiere el cliente se encuentra en la especificación, la validación consiste en asegurar lo que el cliente de verdad quiere.

Este responde a la pregunta **¿Estamos construyendo el producto correcto?**

8. ¿Cual es el objetivo de la VyV?

El objetivo de la verificación y validación consiste en establecer la seguridad de que el sistema software está hecho para un propósito. Esto significa que el sistema es bueno para su uso pretendido.

9. ¿De qué depende el nivel de confianza?

El nivel de confianza depende:

- **Función del software:** El grado en el que un sistema sea crítico para la organización.
- **Expectativas del usuario:** El grado de aceptación a las fallas por parte del usuario final.

- **Entorno de mercado:** La cantidad de competidores, el precio que está dispuesto a pagar sus clientes y la agenda requerida para entregar dicho sistema.

10. ¿En qué momento se realiza el V y V?

La **verificación y validación** (V y V) normalmente se realiza después del proceso de **implementación**. El programa debe ser comprobado para asegurar que se cumple la especificación y entrega la funcionalidad esperada por los clientes. V y V es el nombre que se le da al proceso de analizar y probar lo anteriormente dicho.

11. ¿Qué tipos de análisis y comprobaciones existen dentro del V y V?

Dentro del V y V existen dos aproximaciones complementarias para el análisis y comprobación del sistema:

- Las **inspecciones del software** son aquellas que analizan y comprueban las representaciones del sistema como los diagramas de flujo, los documentos requeridos, el código fuente, etc. Estas pruebas pueden ser complementadas con algún tipo de automatización. Estas **técnicas** son **estáticas** en la V y V porque no se necesita ejecutar el software en computadoras.
- Las **pruebas del software** implica ejecutar una implementación del software con datos de prueba. Se examinan las salidas del software y su entorno operacional para comprobar que funcione tal y como se requiere. Estas son **técnicas dinámicas**.

12. ¿La validación se puede probar con técnicas estáticas?

No, las técnicas estaticas sirven para comprobar que el sistema cumpla con su especificación de requerimientos pero no sirve para ver si el software es operacionalmente útil

13. ¿Que tipos de pruebas existen?

Existen dos tipos de pruebas distintas que se utilizan en diferentes etapas:

- Las **pruebas de validación:** Intenta demostrar que el software es el que el cliente quiere. Se pueden utilizar pruebas estadísticas para probar el rendimiento y la fiabilidad de los programas. Además de comprobar cómo trabaja en ciertas condiciones operacionales.

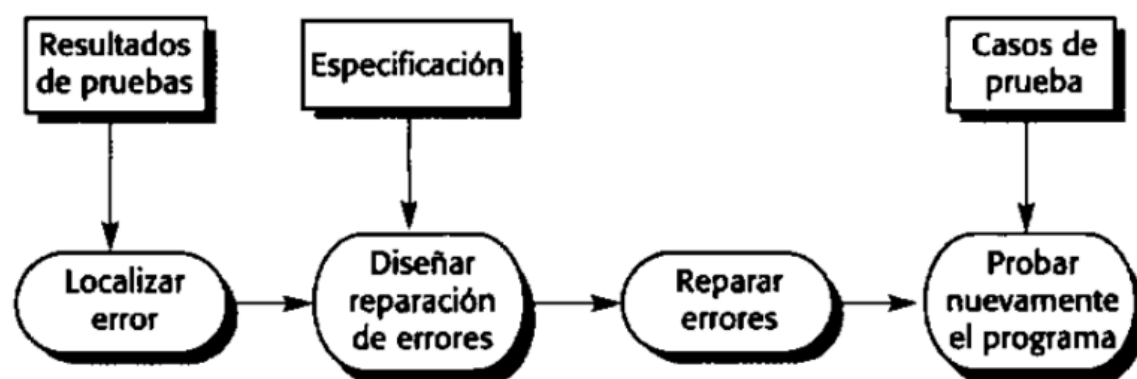
- Las **pruebas de defectos**: Intentan revelar defectos en el sistema en lugar de simular su uso operacional. El objetivo es hallar inconsistencias entre un programa y su especificación.

14. ¿Cuál es la diferencia entre V y V y depuración?

La principal diferencia entre estos procesos es el objetivo del mismo:

- La **verificación y validación** intenta establecer la existencia de defectos en el sistema software.
- La **depuración** es un proceso que localiza y corrige estos defectos.

El proceso de la depuración se ve así:



15. ¿Cómo se planifica? ¿Para que sirve el plan de pruebas?

La verificación y validación es un proceso caro por lo que se debe hacer una planificación cuidadosa para obtener el máximo provecho de las inspecciones y pruebas y controlar los costes de este proceso. Esta planificación se debe dar en etapas tempranas del proceso.

Se tiene que decidir un equilibrio entre aproximaciones estáticas y dinámicas de lo que se debería pensar en estándares y procedimientos para las inspecciones y pruebas del software y definir el plan de pruebas del software.

Los **planes de pruebas** ayudan a los gestores a asignar recursos y estimar el calendario de las pruebas, también son de utilidad para los ingenieros de software implicados en el diseño y la realización de las pruebas. Brinda un panorama al personal técnico de las pruebas técnicas.

Su estructura se compone de:

- **El proceso de prueba:** Una descripción de las principales fases del proceso de prueba.
- **Trazabilidad de requerimientos:** Los usuarios son los más interesados en que el sistema satisfaga sus requerimientos y las pruebas deberían planificarse para que todos los requerimientos se prueben individualmente.
- **Elementos probados:** Deberían especificarse los elementos del proceso del software que tienen que ser probados.
- **Calendario de pruebas:** Un calendario de todas las pruebas y la asignación de recursos para este calendario se enlaza, obviamente, con la agenda general del desarrollo del proyecto.
- **Procedimiento de registro de las pruebas:** No es suficiente ejecutar simplemente las pruebas; los resultados de las pruebas deben ser registrados sistemáticamente. Debe ser posible auditar el proceso de pruebas para comprobar que se ha llevado a cabo correctamente.
- **Requerimientos de hardware y software:** Esta sección debería determinar las herramientas software requeridas y la utilización estimada del hardware.
- **Restricciones:** En esta sección deberían anticiparse las restricciones que afectan al proceso de pruebas como la escasez de personal.

16. ¿Qué es una inspección? ¿Qué ventajas posee sobre las pruebas?

Las inspecciones de software son un proceso de V y V estático en el que un sistema software se revisa para encontrar errores, omisiones y anomalías. Estas pueden inspeccionarse cualquier representación legible del sistema y al inspeccionarse se debe utilizar el conocimiento del sistema, su dominio de aplicación y el lenguaje de programación

Estas poseen tres ventajas sobre las pruebas:

- Al ser estático si se encuentra un error no hay que preocuparse de las interacciones entre errores.
- No necesita estar completo el software para inspeccionarse sin costo extra.
- Busca cumplir atributos de calidad como el cumplimiento de los estándares, portabilidad y mantenibilidad. Puede buscar ineficiencias, algoritmos no adecuados, etc.

17. ¿Cómo es el proceso de inspección de software?

La inspección de programas es un proceso formal que se realiza en equipo con las cualidades necesarias. Se sugieren seis roles:

Autor o propietario	El programador o diseñador responsable de generar el programa o documento. Responsable de reparar los defectos descubiertos durante el proceso de inspección.
Inspector	Encuentra errores, omisiones e inconsistencias en los programas y documentos. También puede identificar cuestiones más generales que están fuera del ámbito del equipo de inspección.
Lector	Presenta el código o documento en una reunión de inspección.
Secretario	Registra los resultados de la reunión de inspección.
Presidente o moderador	Gestiona el proceso y facilita la inspección. Realiza un informe de los resultados del proceso para el moderador jefe.
Moderador jefe	Responsable de las mejoras del proceso de inspección, actualización de las listas de comprobación, estándares de desarrollo, etc.

Figura 22.5 Roles en el proceso de inspección.

Las personas del equipo pueden adoptar varios roles de forma que varía el tamaño en cada inspección. Como mínimo debe haber alguien de pruebas, usuario final y gestión de calidad.

Antes de que empiece una inspección del software se debe tener:

- Una especificación precisa del código a inspeccionar.
- Los miembros deben tener claro los estándares de la organización.
- Se distribuye una versión compilada y actualizada del código a todos los miembros del equipo.

El proceso es el siguiente:

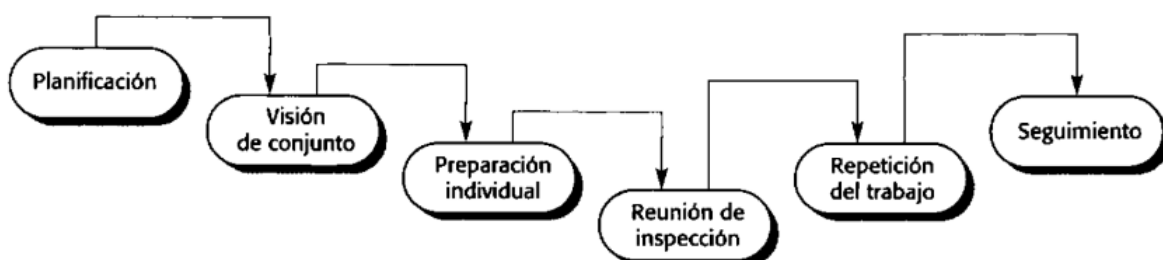


Figura 22.6 El proceso de inspección.

- **Planificación:** El moderador es el responsable de esta área. Selecciona un equipo de inspección, organiza una sala de reuniones y asegura que el material a inspeccionar y sus especificaciones estén completas.
- **Visión de conjunto:** El programa se presenta al equipo y el autor del código describe lo que el programa debería hacer.

- **Preparación individual:** Cada miembro del equipo estudia la especificación y el programa y busca defectos en código.
- **Reunión de inspección:** Esta no debe durar más de dos horas y se debe centrar en buscar en la detección de defectos, cumplimiento de los estándares y programación de baja calidad. No se deben sugerir como repararse ni recomendar cambios. Se utilizan **listas de comprobaciones** de errores que son una serie de preguntas de si y no para verificar que se cumpla todos los criterios.
- **Repetición del trabajo:** El autor del código deberá realizar los cambios para corregir los problemas identificados. Modificación del documento o código según las recomendaciones, y preparación para una posible re-inspección si es necesario.
- **Seguimiento:** El moderador deberá decidir si se requiere una reinspección del código en caso de que no, el programa es aprobado por el moderador para su entrega. Se revisan las correcciones realizadas, cierre del proceso de inspección, y documentación de los resultados finales.

18. ¿Que es un analisis estatico automatico?

Un **análisis estático automático** es una herramienta software que escanea el código fuente y detecta los posibles defectos, anomalías, malas prácticas, etc. Estos pueden detectar si las sentencias están bien formadas, hacer inferencias sobre el flujo de control del programa y calcular el conjunto de todos los posibles valores para los datos del programa.

19. ¿Qué son los métodos formales?

Los métodos formales hacen referencia a una representación matemática del software, se suele ocupar del análisis matemática de la especificación.

20. ¿Qué es el desarrollo de software de sala limpia?

El desarrollo de software de sala limpia es una filosofía del desarrollo del software que utiliza métodos formales para soportar inspecciones del software rigurosas. Su objetivo es obtener un software con 0 defectos

El proceso es el siguiente

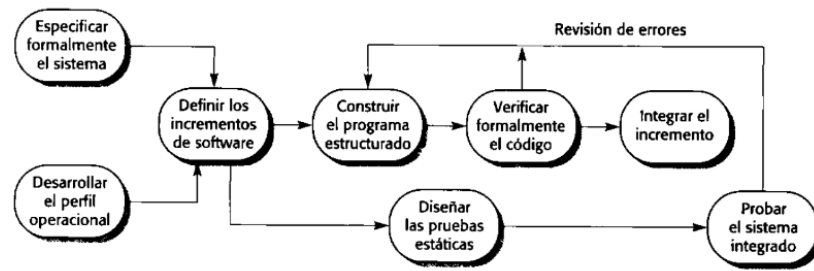


Figura 22.10 El proceso de desarrollo de Sala Limpia.

Unidad 3 - Revisiones

Libros:

Cap 15 -  Roger S Pressman - Ingeniería del software _ un enfoque práctico (7...

Cap 22 -  Ian.Sommerville-7maEdicion-Ingenieria.de.Software.pdf

Preguntontas

1. ¿Qué son las revisiones? ¿Para qué sirven?

Las **revisiones del software** son un filtro para el proceso de desarrollo de software con el fin de detectar errores y defectos en las distintas etapas para poder eliminarlos. Las revisiones se utilizan para resaltar las mejoras de un producto, confirmar aquellas partes que no necesitan correcciones y realizar el trabajo técnico de calidad más uniforme.

2. ¿Cuál es el objetivo principal de las revisiones técnicas?

El objetivo principal de las revisiones técnicas es encontrar errores durante el proceso a fin de que no se conviertan en defectos después de liberar el software. El beneficio de las revisiones técnicas es el descubrimiento temprano de los errores, de modo que no se propaguen a la siguiente etapa del proceso del software.

3. ¿Cuáles son las consecuencias de los defectos del software?

Los defectos son errores que se encuentran con el producto ya liberado, esto tiene grandes consecuencias económicas ya que arreglar un error en una etapa de producción es mucho más costoso que en una etapa de elaboración y puede perjudicar la imagen de la empresa, la confianza, etc.

4. ¿Qué métricas se utilizan?

Se puede medir los distintos tipos de esfuerzo como el de preparación, evaluación y repetición, todas estas poseen de unidad horas hombre.

Otra métrica son los errores menores y mayores que fueron detectados.

5. ¿Qué es el espectro de formalidad?

El espectro de formalidad es el grado de formalidad que se debe emplear según el tipo de proyecto que se esté elaborando, no es lo mismo revisar un sistema crítico a uno que no lo sea, se espera mucho más rigurosidad sobre el trabajo realizado.

6. ¿Qué es el equipo de revisión?

El equipo de revisión es un equipo compuesto por varias personas del desarrollo, tester, etc cuando una revisión necesita un enfoque más riguroso o más formal.

Cada integrante tendrá un rol específico, después cada revisor tendrá acceso al trabajo y los revisará. Se debe planificar las tareas de cada uno y los resultados deben quedar registrados de manera formal.

7. ¿Cuáles son las categorías de revisiones?

Las revisiones técnicas se pueden separar en revisiones informales y formales.

8. ¿Qué es una revisión informal? ¿Que es una lista de revisión?

Las **revisiones informales** consisten en una revisión simple que se realiza sobre algún producto obtenido en el desarrollo de software pero que no requiere de un grado de formalidad muy alto. Por ejemplo, mientras estás desarrollando le puedes preguntar a tu compañero de equipo para revisar si le parece bien la forma que desarrollaste tu código o tu diseño.

Estas revisiones se realizan sin ningún tipo de planificación por lo que no son muy eficaces pero aun así sirven.

Las **listas de revisiones** sirven para mejorar la eficacia de este tipo de revisiones y son un conjunto de preguntas generales para guiar a los revisores y así verificar el software

9. ¿Qué es una revisión formal?

Una revisión técnica formal es una actividad del control de calidad que es realizada por los ingenieros de software. Sus objetivos son

- Descubrir errores en funcionamiento, lógica o implementación de cualquier representación del software.
- Verificar que el software que se revisa cumple con los requerimientos (verificación).
- Garantizar que el software está representado de acuerdo con los estándares predefinidos (estilos de codificación).
- Obtener software desarrollado de manera uniforme. (De manera más similar)
- Hacer proyectos más manejables.

Una ventaja de estos métodos es que sirve como capacitación ya que muestra los diferentes enfoques que adoptan los ingenieros más avanzados sobre las diferentes etapas.

Cada Revisión Técnica Formal (RTF) se realiza como una reunión y esta debe planificarse, controlarse y ejecutarse de manera organizada para tener éxito.

10. ¿Cómo son las reuniones RFT? ¿Qué es un reporte técnico formal?

Las reuniones de revisión deben cumplir con estas restricciones:

- Deben participar entre 3 a 5 personas.
- Debe existir una preparación previa pero no más de dos horas de trabajo.
- La duración de la reunión debe ser de al menos dos horas.

El procedimiento es el siguiente:

- El productor informa al líder del proyecto que terminó y que se necesita una revisión.
- El líder del proyecto contacta al líder de la revisión.
- El líder de revisión entrega copias a los revisores y programa una reunión, normalmente, para el día de mañana.
- A esta reunión acuden los revisores, los líderes y el productor.
- Un revisor adopta el rol de secretario.
- El productor cuenta lo que hizo.
- Los revisores van comentando con el análisis que debieron hacer. Si se encuentran errores, el secretario toma nota de ello.
- Al terminar la revisión, todos los asistentes deben decidir si se acepta el producto, si no se acepta por errores graves y si se acepta parcialmente ya que posee errores menores pero funciona.

Una vez finalizada esta reunión se obtiene un documento denominado **lista de pendientes de la revisión** y se elabora un **reporte técnico formal de la revisión** que responde a tres preguntas ¿Qué fue lo que se revisó? ¿Quién lo revisó? ¿Cuáles fueron los descubrimientos y las conclusiones?

La lista tiene como fin dos propósitos, identificar áreas de problemas en el producto y servir como lista de verificación de acciones que guíe al productor cuando haga las correcciones.

11. ¿Cuáles son los lineamientos para realizar una revisión?

Los lineamientos que se deben seguir son:

- **Revise el producto, no al productor.** Una RTF involucra personas y sus egos. Si se lleva a cabo en forma adecuada, la RTF debe dejar en todos los participantes una sensación de logro. Si se efectúa de modo inapropiado, adopta un aire inquisitorial. Los errores deben señalarse en forma amable; el

tono de la reunión debe ser relajado y constructivo; el trabajo no debe apenar o menospreciar a nadie. El líder de la revisión debe conducir la reunión en tono y actitud apropiados y debe detenerla de inmediato si se sale de control.

- **Establezca una agenda y sígala.** Una de las fallas clave de las reuniones de todo tipo es la dispersión. Una RTF debe mantenerse encarrilada y dentro del programa. El líder de la revisión tiene la responsabilidad de que así sea y no debe sentir temor de llamar al orden a las personas cuando se dispersen.
- **Limite el debate y las contestaciones.** Cuando el revisor plantee un asunto, quizá no haya acuerdo universal acerca de su efecto. En vez de perder tiempo en debatir la cuestión, ésta debe registrarse para discutirla después.
- **Enuncie áreas de problemas, pero no intente resolver cada uno.** Una revisión no es una sesión para resolver problemas. Es frecuente que la solución de un problema la obtenga el productor, solo o con ayuda de otra persona. La solución de los problemas debe posponerse para después de la reunión de revisión.
- **Tome notas por escrito.** A veces es buena idea que el secretario tome notas en un pizarrón a fin de que la redacción y prioridades sean evaluadas por los demás revisores a medida que la información se registra. De manera alternativa, pueden tomarse notas directamente en una computadora.
- **Limite el número de participantes e insista en la preparación previa.** Dos cabezas piensan más que una, pero 14 no son necesariamente mejor que 4. Mantenga limitado el número de personas involucradas. Sin embargo, todos los miembros del equipo de revisión deben prepararse. El líder de la revisión tiene que solicitar comentarios por escrito (lo que proporciona un indicador de que el revisor ha inspeccionado el material).
- **Desarrolle una lista de verificación para cada producto que sea probable que se revise.** Una lista de verificación ayuda al líder del proyecto a estructurar la RTF y a cada revisor a centrarse en los aspectos importantes. Deben desarrollarse listas para los productos del análisis, el diseño, el código e incluso las pruebas.
- **Asigne recursos y programe tiempo para las RTF.** Para que las revisiones sean eficaces, deben programarse como tareas del proceso de software. Además, debe programarse tiempo para hacer las inevitables modificaciones que ocurrirán como resultado de la RTF.

- **Dé una capacitación significativa a todos los revisores.** Para que una revisión sea eficaz, todos los revisores deben recibir cierta capacitación formal. Ésta debe hacer énfasis tanto en aspectos relacionados con el proceso como en el lado de la psicología humana de la revisión.
- **Revise las primeras revisiones.** Volver a revisar puede ser benéfico para descubrir problemas con el proceso de revisión en sí mismo. El primer producto por revisar deben ser los lineamientos de la revisión.

Unidad 4 - Pruebas de software

Libros:

Cap 17 -  Roger S Pressman - Ingeniería del software _ un enfoque práctico (7...

Cap 23 -  Ian.Sommerville-7maEdicion-Ingenieria.de.Software.pdf

Preguntontas

1. ¿Cómo es un proceso de pruebas?

Un proceso de prueba en sistemas grandes es comenzar con las pruebas de unidad (pruebas hacia funciones u objetos), después se procede a probar la integración entre estos componentes y para finalizar, el cliente comprueba que el sistema funcione tal y como se especificó mediante pruebas de validación.

En sistemas medianos y chicos se simplifica este proceso por lo que utilizan menos pruebas

2. ¿Cuáles son las dos actividades fundamentales de pruebas?

Las dos actividades fundamentales de pruebas son la prueba de componentes, es decir, que prueba las partes del sistema, y las pruebas de sistema, que prueban el sistema como un todo

3. ¿Qué objetivos posee el proceso de pruebas?

Los objetivos que posee el proceso de pruebas son:

- Demostrar al desarrollador y al cliente que el software satisface sus requerimientos
- Descubrir defectos en el software en que el comportamiento de este es incorrecto, no deseable o no cumple con su especificación.

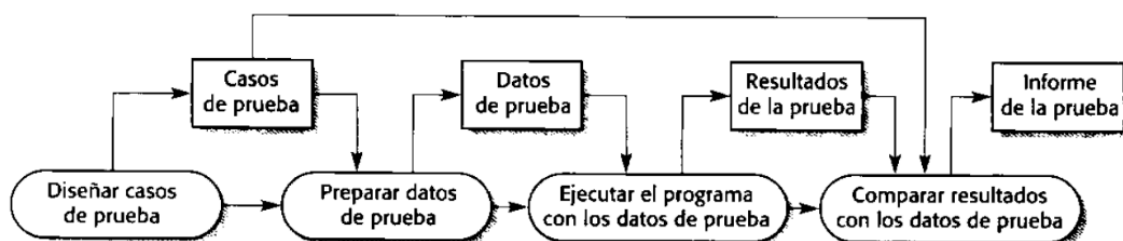
4. ¿Cuando una prueba tiene éxito?

Para las pruebas de validación el éxito se define como que el sistema funcione correctamente y para las pruebas de defectos el éxito se define como que demuestra un defecto del software que hace que funcione incorrectamente

5. ¿Las pruebas pueden demostrar que el software está libre de defecto?

No, según Dijkstra “Las pruebas solo pueden demostrar la presencia de errores, no su ausencia”.

6. ¿Cómo es el modelo general del proceso?



Los **casos de prueba** son especificaciones de las entradas para la prueba y la salida esperada del sistema, además de afirmar lo que se está probando.

Los **datos de prueba** son las entradas que han sido ideadas para probar el sistema (a veces se puede generar automáticamente).

Los casos de prueba no se pueden automatizar ya que solo el desarrollador puede predecir la salida.

7. ¿Se debe probar todo?

No, probar cada secuencia del programa es imposible por lo que las pruebas se deberán basar en un subconjunto de todos los posibles casos de prueba, por lo general, las empresas deberán definir unas políticas que ayuden al programador a elegir el subconjunto previamente dicho.

8. ¿Qué son las pruebas del sistema?

Las **pruebas del sistema** implica integrar dos o más componentes que implemente funciones del sistema o características y que se pruebe cómo funciona este sistema integrado.

Estas pruebas se pueden separar en **dos fases**:

- Pruebas de integración.
- Pruebas de entrega.

9. ¿Qué son las pruebas de integración? ¿Cómo es el proceso de integración? ¿Qué son las pruebas de regresión?

Las **pruebas de integración** son aquellas pruebas en las que el equipo tiene acceso al código fuente y su principal prioridad es encontrar defectos. La dificultad de este tipo de pruebas es localizar los errores

El **proceso de integración del sistema** consiste en construir este a partir de sus componentes y probarlo para encontrar problemas por la integración de los

componentes. Los componentes se pueden dividir en **componentes comerciales**, **componentes reutilizables** que han sido adaptados o **componentes nuevos** desarrollados. Normalmente se usan los 3 y lo que hacen este tipo de pruebas es que comprueben que funcionen juntos, son llamados correctamente y transfieren los datos correctos en el tiempo preciso, etc.

La **integración** del sistema puede ser de manera **descendente** (Primero se desarrolla el esqueleto del sistema y después se van añadiendo componentes) o **ascendente** (Se hacen componentes de servicio común y se añaden componentes funcionales). Se usan los dos.

Las **pruebas de regresión** consisten en volver a ejecutar un conjunto existente de pruebas. No son viables pero si se programan las pruebas como en JUnit se pueden volver a ejecutar sin ningún tipo de problema.

10. ¿Qué son las pruebas de entrega?

Las **pruebas de entregas** son el proceso de probar una entrega del sistema que será distribuida a los clientes. Su principal objetivo es incrementar la confianza del cliente en el que el sistema satisface sus requerimientos. Cuando estas pruebas se realizan, deberían intentar romper el software eligiendo los casos de pruebas más propensos a generar fallos de ejecución.

Este tipo de pruebas son un **proceso de pruebas de caja negra** en la que las pruebas se derivan a partir de la especificación del sistema. Se considera al sistema como una caja negra cuyo comportamiento sólo se puede determinar probando sus entradas y salidas, también se llaman **pruebas funcionales**.

Para validar los requerimientos el mejor tipo de prueba son las pruebas basadas en escenarios que plantean varios escenarios o historia de usuario y se hacen los casos de pruebas a partir de este. Para cada prueba se debe diseñar entradas validas e invalidas y que generen salidas validas e invalidas.

Si se utilizó los casos de uso y diagramas de uso pueden ayudarte a preparar las pruebas de entrega

11. ¿Cuáles son las pruebas de rendimiento?

Una vez que se finalizó la integración, se puede probar diferentes propiedades emergentes como rendimiento, fiabilidad, etc.

Las **pruebas de rendimiento** se diseñan para asegurar que el sistema pueda procesar su carga esperada. Para verificar que cumpla con los requerimientos se debe hacer un perfil operacional, es decir, en donde se concentra la mayor cantidad de operaciones y de ahí definir las pruebas.

Para encontrar defectos se debe diseñar alrededor de los límites del sistema. Estas pruebas buscan estresar al sistema por lo que se le llama **pruebas de estrés**.

Tiene dos funciones:

- Probar el comportamiento de fallo de ejecución del sistema. Busca evitar que en caso de que falle que no se rompa todo.
- Sobrecargan el sistema y pueden provocar que se manifiesten defectos que normalmente no serían descubiertos.

12. ¿Qué son las pruebas de componentes o de unidad?

Las **pruebas de componentes o de unidad** son el proceso de probar los componentes individuales de un sistema. Su prioridad es encontrar la mayor cantidad de defectos. Los desarrolladores son los que se encargan de probarlo.

Los tipos de componentes que se pueden probar son:

- Funciones o métodos individuales dentro de un objeto.
- Clases de objetos que tienen varios atributos y métodos.
- Componentes compuestos formados por diferentes objetos o funciones. Estos tienen una interfaz definida para acceder a su funcionalidad.

Las pruebas a funciones o métodos individuales dentro de un objeto se hacen solo con llamadas a estas rutinas con diferentes parámetros de entrada.

Las clases de objetos se deben diseñar las pruebas para proporcionar cobertura para todas las características del objeto, es decir, se debe incluir:

- Las pruebas aisladas de todas las operaciones asociadas con el objeto.

- La asignación y consulta de todos los atributos asociados con el objeto.
- Ejecutar el objeto en todos sus posibles estados. Simular todos los eventos que provoquen cambios

13. ¿Cuáles son las pruebas de interfaces?

A veces, muchos componentes no son funciones u objetos sino que son componentes formados por varios objetos comunicados entre sí a través de una interfaz. Las pruebas se deben realizar sobre esta interfaz porque si probamos cada componente por separado y el problema se encuentra en la interfaz no servirá.

Existen distintos tipos de interfaces:

1. *Interfaces de parámetros.* Son interfaces en las que los datos, o algunas veces referencias a funciones, se pasan de un componente a otro en forma de parámetros.
2. *Interfaces de memoria compartida.* Son interfaces en las que un bloque de memoria se comparte entre los componentes. Los datos se colocan en la memoria por un subsistema y son recuperados desde aquí por otros subsistemas.
3. *Interfaces procedurales.* Son interfaces en las que un componente encapsula un conjunto de procedimientos que pueden ser llamados por otros componentes. Los objetos y los componentes reutilizables tienen esta forma de interfaz.
4. *Interfaces de paso de mensajes.* Son interfaces en las que un componente solicita un servicio de otro componente mediante el paso de un mensaje. Un mensaje de retorno incluye los resultados de la ejecución del servicio. Algunos sistemas orientados a objetos tienen esta forma de interfaz, así como los sistemas cliente-servidor.

Los tipos de errores que se pueden encontrar son:

- **Mal uso de la interfaz.** Un componente llama a otro componente y comete un error en la utilización de la interfaz. Típico en interfaces de parámetros porque los parámetros no corresponden a su tipo o una cantidad diferente a la que se espera.
- **No comprensión de la interfaz.** No se cumple con lo requerido para que se cumpla el comportamiento, un ejemplo sería pasar un vector desordenado a una búsqueda binaria.
- **Errores temporales.** Sucede en sistemas de tiempo real que utilizan la interfaces de memoria compartida o paso de mensaje. El productor de los datos y el consumidor pueden trabajar a diferentes velocidades. Si no se

tiene cuidado en el diseño el consumidor puede agarrar datos desactualizados.

He aquí algunas guías generales para las pruebas de interfaz:

1. Examinar el código a probar y listar explícitamente cada llamada a un componente externo. Diseñar un conjunto de pruebas en donde los valores de los parámetros para los componentes externos están en los extremos de sus rangos. Es bastante probable que estos valores extremos revelen inconsistencias en la interfaz.
2. En los lugares en los que se pasan punteros a través de una interfaz, siempre probar la interfaz con parámetros de punteros nulos.
3. Cuando se llama a un componente a través de una interfaz procedural, diseñar pruebas que hagan que el componente falle. Realizar suposiciones de fallos de ejecución erróneas es una de las malas interpretaciones de especificación más comunes.
4. Utilizar las pruebas de estrés, tal y como se indicó en la sección previa, en los sistemas de paso de mensajes. Diseñar pruebas que generen muchos más mensajes de los que probablemente ocurran en la práctica. Los problemas temporales se detectan de esta manera.
5. Cuando varios componentes interactúan a través de memoria compartida, diseñar pruebas que varíen el orden en el que se activan estos componentes. Estas pruebas pueden revelar suposiciones implícitas hechas por el programador sobre el orden en el que los datos compartidos son producidos y consumidos.

14. ¿Cómo se diseñan los casos de prueba?

Este diseño es una parte de los dos tipos de pruebas ya sea de componentes o de sistemas. Los **casos de pruebas** son aquellas entradas y salidas esperadas que se necesitan para probar el sistema. El **objetivo** del proceso de diseño es crear un conjunto de casos de pruebas que sean efectivos al descubrir defectos y mostrar que el sistema satisface sus requerimientos.

Para diseñar se elige una característica del sistema que se está probando. Después se selecciona un conjunto de entradas que ejecutan dicha característica, se documenta las salidas o rangos esperados y si es posible se diseña una prueba automatizada que pruebe las salidas reales con las esperadas.

15. ¿Qué tipo de pruebas se necesitan para los casos de pruebas?

Para diseñar casos de pruebas existen varias aproximaciones:

- **Pruebas basadas en requerimientos.**

Los casos de pruebas se diseñan según los requerimientos del sistema, se utiliza en las pruebas de sistema ya que un requerimiento puede involucrar varias partes del

mismo. Para cada requerimiento, se identifican casos de pruebas que demuestren que el sistema satisface el requerimiento.

Como se establece en la ingeniería de requerimientos es que los requerimientos se deben poder probar por lo que se podría diseñar un caso de prueba para que alguien pueda comprobar que se cumple. Este tipo de pruebas son más que nada de validación ya que no busca defectos sino que se compruebe que el sistema implementa los requerimientos solicitados

- **Pruebas de particiones.**

Se identifican particiones de entrada y salida y se diseñan pruebas para que el sistema ejecute entradas de todas las particiones y genere salidas de todas las particiones. En sí, las particiones son un grupo de datos que tienen características en comunes como todos los números negativos y así.

En este contexto se suele usar mucho el término **clase** para hacer referencia a una partición, no olvidarse.

Normalmente los programas se comportan de manera similar para todos los miembros de una partición. Por esta razón, a las particiones se les denomina **particiones equivalentes o dominio**.

Una forma sistemática al diseño de casos de pruebas se basa en identificar todas las particiones que existen y los casos de prueba se diseñan para que las entradas o salidas pertenezcan a estas particiones. Este tipo de pruebas se suele usar en pruebas de componentes como de sistemas.

Para elegir qué pruebas debemos realizar, una buena práctica es elegir casos de pruebas en los límites de las particiones junto con algunos en el medio

Una aclaración, las entradas ya sea válidas o inválidas pueden formar particiones de equivalencia.

- **Pruebas estructurales.**

Se utiliza el conocimiento de la estructura del programa para diseñar pruebas que ejecuten todas las partes del programa. Esta aproximación se le suele llamar

pruebas de **caja blanca**, de **caja de cristal** o de **caja transparente** para distinguirla de las de caja negra (que se necesita el sistema total y no conocer la estructura del programa)

Las **pruebas de camino** son una estrategia de pruebas estructurales cuyo objetivo es probar cada camino de ejecución independiente en un componente o programa. Como cada camino es independiente, cada sentencia del componente se debe haber ejecutado al menos una vez.

Este tipo de pruebas se utilizan en las pruebas de componentes ya que solo es factible en este ámbito.

Para realizar una prueba de camino se debe realizar un **grafo de flujo** del programa que es un modelo esqueleto de todos los caminos del programa. Los nodos son decisiones y las aristas son el flujo de control. Un camino independiente se define como el recorrido de al menos una nueva arista del grafo. El **objetivo de las pruebas de camino** es que cada camino independiente se ejecute al menos una vez. En términos de programa es ejecutar una o más condiciones nuevas.

16. ¿Qué es la automatización de pruebas?

La **automatización de pruebas** hace referencia a programar las pruebas de tu sistema para que la puedas ejecutar. Se establecen las entradas y salidas esperadas y se debe corresponder con la salida real.

Un **banco de pruebas** del software es un conjunto integrado de herramientas para soportar el proceso de pruebas. Un banco de trabajo, además de soportar la ejecución automática de las pruebas puede incluir herramientas para simular otras partes del sistema y generar datos de pruebas.

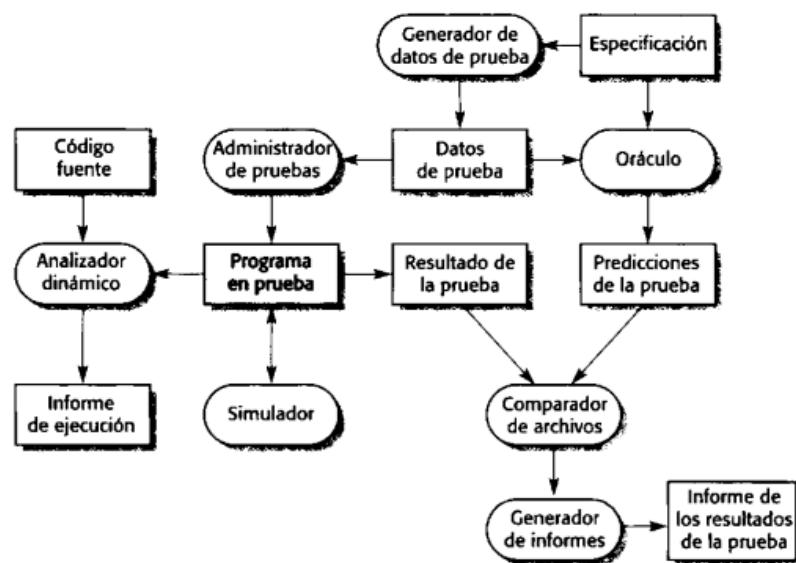


Figura 23.17
Un banco de trabajo de pruebas.

1. *Gestor de pruebas.* Gestiona la ejecución de las pruebas del programa. El gestor de pruebas mantiene un registro de los datos de las pruebas, resultados esperados y facilidades del programa que han sido probadas. Los marcos de trabajo automatizados tales como JUnit son ejemplos de gestores de pruebas.
2. *Generador de datos de prueba.* Genera datos de prueba para el programa a probar. Esto puede conseguirse seleccionando datos de una base de datos o utilizando patrones para generar datos aleatorios de forma correcta.
3. *Oráculo.* Genera predicciones de resultados esperados de pruebas. Los oráculos pueden ser versiones previas del programa o sistemas de prototipos. Las pruebas *back-to-back* (estudiadas en el Capítulo 17) implican ejecutar el oráculo y el programa a probar en paralelo. Las diferencias entre sus salidas son resaltadas.
4. *Comparador de ficheros.* Compara los resultados de las pruebas del programa con los resultados de pruebas previos e informa de las diferencias entre ellos. Los comparadores se utilizan en pruebas de regresión en las que se comparan los resultados de ejecutar diferentes versiones. Cuando se utilizan pruebas automatizadas, los comparadores pueden ser llamados desde las mismas pruebas.
5. *Generador de informes.* Proporciona la definición de informes y facilidades de generación para los resultados de las pruebas.
6. *Analizador dinámico.* Añade código a un programa para contar el número de veces que se ha ejecutado cada sentencia. Después de las pruebas, se genera un perfil de ejecución que muestra cuántas veces se ha ejecutado cada sentencia del programa.
7. *Simulador.* Se pueden utilizar diferentes tipos de simuladores. Los simuladores de la máquina objetivo simulan la máquina sobre la que se ejecuta el programa. Los simuladores de interfaces de usuario son programas conducidos por *scripts* que simulan múltiples interacciones de usuarios simultáneas. Utilizar simuladores para Entrada/Salida implica que el comportamiento temporal de la secuencia de las transacciones es repetible.

17. ¿Qué es una aproximación?

Una "aproximación", en este contexto, se refiere a la estrategia o método adoptado para llevar a cabo las pruebas.

18. ¿Que es una estrategia de pruebas de software? ¿Qué debe incorporar?

Una **estrategia de pruebas de software** es una guía que describe los pasos que deben realizarse como parte de la prueba, cuando se planea y se llevan a cabo esos pasos, y cuánto esfuerzo, tiempo y recursos se requerirán.

Para este tipo de estrategias debe incorporar una planificación de las pruebas, el diseño de los casos de prueba, la ejecución de las pruebas y la recolección y evaluación de los resultados.

19. ¿Qué es una plantilla en este contexto? ¿Qué tipo de pruebas debe incluir una estrategia de pruebas?

Como se conoce las pruebas son un conjunto de actividades que pueden planearse por adelantado y realizarse de manera sistemática. Por lo que se debe definir una **plantilla** para las pruebas del software que sería un conjunto de pasos que incluye métodos de prueba y técnicas de diseño de casos de prueba específicos.

Algunas características generales de las plantillas de pruebas son:

- Para realizar una prueba efectiva, debe realizar revisiones técnicas efectivas (capítulo 15). Al hacerlo, eliminará muchos errores antes de comenzar la prueba.
- La prueba comienza en los componentes y opera "hacia afuera", hacia la integración de todo el sistema de cómputo.
- Diferentes técnicas de prueba son adecuadas para distintos enfoques de ingeniería de software y en diferentes momentos en el tiempo.
- Las pruebas las realiza el desarrollador del software y (para proyectos grandes) un grupo de prueba independiente.
- Prueba y depuración son actividades diferentes, pero la depuración debe incluirse en cualquier estrategia de prueba.

Las estrategias de pruebas deben incluir pruebas de bajo nivel que son necesarias para verificar que pequeñas partes del código funcionan correctamente. También deben incluir pruebas de alto nivel las cuales validan las principales funciones del sistema a partir de los requerimientos del cliente.

20. ¿Cuáles son las actividades SQA?

La verificación se refiere al conjunto de tareas que garantizan que el software implementa correctamente una función específica.

La validación es un conjunto diferente de tareas que aseguran que el software que se construye sigue los requerimientos del cliente.

La definición del V y V abarca muchas actividades de garantía de calidad del software. La verificación y validación incluye una amplia gama de actividades SQA ("Software Quality Assurance" o en español Aseguramiento de la Calidad del Software):

- Revisiones técnicas.
- Auditorías de calidad y configuración.
- Monitoreo de rendimiento.
- Simulación.
- Estudio de factibilidad.
- Revisión de documentación.
- Revisión de base de datos.
- Análisis de algoritmos.
- Pruebas de desarrollo.
- Pruebas de usabilidad.
- Pruebas de calificación.
- Pruebas de aceptación.
- Pruebas de instalación.

La calidad se incorpora en el software a lo largo de todo el proceso de ingeniería del software.

21. ¿Qué circunstancias pueden afectar las pruebas de un desarrollador? ¿Qué es el GPI?

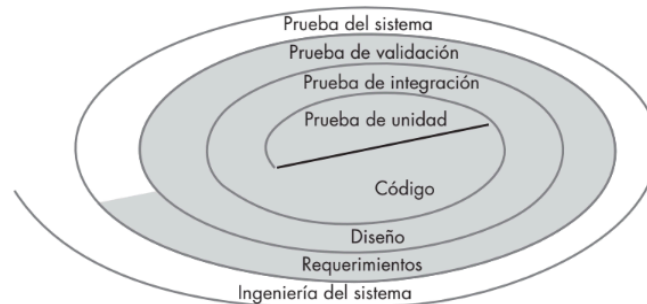
El apego emocional que posee el desarrollador para intentar probar o derrumbar el software tiene un efecto negativo a la hora de probarlo. Esto no significa que el

desarrollador no pueda probar su software, en sí debe participar en realizar las pruebas de componentes o de unidad y en algunos casos las de integración.

Una vez que la arquitectura de software está completa se deberá involucrar el **grupo de pruebas independientes** que tendrán como objetivo remover los problemas inherentes que se encuentran asociados con dejar al constructor probar lo que construyó.

22. ¿Cómo se separan las pruebas? Explique

FIGURA 17.1
Estrategia de pruebas



Como se ve en la figura, es un espiral donde se muestra el proceso de desarrollo de software y en qué momento se comienzan a hacer las pruebas.

Las **pruebas de unidad** son aquellas que se concentran en cada unidad del software como se implementó en el código.

Las **pruebas de integración** se enfocan en el diseño y construcción de la arquitectura del software. Estas abordan los conflictos asociados con los problemas duales de verificación y construcción de programas.

Las **pruebas de validación** son aquellas que validan los requerimientos establecidos como parte de su modelado se validan mediante la confrontación con el software que se construyó. A su vez, proporciona la garantía final de que el software cumple con todos los requerimientos informativos, funcionales, de comportamiento y de rendimiento.

Las **pruebas de sistema** son aquellas que el software y otros elementos del sistema se prueban como un todo. Además, verifica que todos los elementos se mezclan de manera adecuada y que se logra el funcionamiento/rendimiento global del sistema.

Al ver el proceso desde un punto de vista procedural, las pruebas son una serie de cuatro pasos que se van implementando de manera secuencial.

23. ¿Qué criterios se deben tener para completar las pruebas?

Una pregunta complicada de responder es en qué momento terminan las pruebas, hay muchos desarrolladores que responden “Nunca terminan las pruebas, solo cambia el probador” y se valen de que cada vez que un usuario ejecuta el programa, este se pone a prueba. Otra forma de responder es cuando se acaba el tiempo o el dinero.

Según el enfoque de la sala limpia se deben utilizar técnicas estadísticas que ejecutan una serie de pruebas de una muestra de una población objetivo. Otros abogan por el uso de modelos estadísticos y teoría de confiabilidad para predecir cuando se completan las pruebas.

24. ¿Qué aspectos se deben tener en cuenta la hora de utilizar una estrategia de prueba?

Los aspectos que si o si se deben tener en cuenta a la hora de utilizar una estrategia de pruebas son:

- **Especificar los requerimientos del producto en forma cuantificable mucho antes de comenzar con las pruebas.** Aunque el objetivo predominante de una prueba es encontrar errores, una buena estrategia de prueba también valora otras características de la calidad, como la portabilidad, el mantenimiento y la facilidad de uso. Esto debe especificarse en una forma medible, de modo que los resultados de las pruebas no sean ambiguos.
- **Establecer de manera explícita los objetivos de las pruebas.** Los objetivos específicos de las pruebas deben enunciarse en términos medibles.

Por ejemplo, la efectividad de las pruebas, su cobertura, el tiempo medio antes de aparecer una falla, el costo por descubrir y corregir defectos, la densidad de defectos restantes o la frecuencia de ocurrencia, y las horas de trabajo de prueba deben enunciarse dentro del plan de la prueba.

- **Entender a los usuarios del software y desarrollar un perfil para cada categoría de usuario.** Los casos de uso que describen el escenario de interacción para cada clase de usuario pueden reducir el esfuerzo de prueba global al enfocar las pruebas en el uso real del producto.
- **Desarrollar un plan de prueba que enfatice “pruebas de ciclo rápido”.** Se recomienda que un equipo de software “aprenda a probar en ciclos rápidos (2 por ciento del esfuerzo del proyecto) de cliente-utilidad al menos la ‘comprobabilidad’ en campo, los incrementos de funcionalidad y/o la mejora de la calidad”. La retroalimentación generada a partir de estas pruebas de ciclo rápido puede usarse para controlar niveles de calidad y las correspondientes estrategias de prueba.
- **Construir software “robusto” que esté diseñado para probarse a sí mismo.** El software debe diseñarse en forma que use técnicas antierrores, es decir, el software debe poder diagnosticar ciertas clases de errores. Además, el diseño debe incluir pruebas automatizadas y pruebas de regresión.
- **Usar revisiones técnicas efectivas como filtro previo a las pruebas.** Las revisiones técnicas pueden ser tan efectivas como probar para descubrir errores. Por esta razón, las revisiones pueden reducir la cantidad del esfuerzo de pruebas que se requieren para producir software de alta calidad.
- **Realizar revisiones técnicas para valorar la estrategia de prueba y los casos de prueba.** Las revisiones de prueba pueden descubrir inconsistencias, omisiones y errores evidentes en el abordaje de las pruebas. Esto ahorra tiempo y también mejora la calidad del producto.
- **Desarrollar un enfoque de mejora continuo para el proceso de prueba.** La estrategia de pruebas debe medirse. Las métricas recopiladas durante las pruebas deben usarse como parte de un enfoque de control de proceso estadístico para la prueba del software.

25. ¿Qué tipo de estrategia se utiliza?

Las diferentes tipos de estrategias que se pueden utilizar son:

- Esperar a que el software esté terminado y ahí recién probarlo. Spoiler no funciona.
- Realizar pruebas diariamente, siempre que se construya alguna parte del sistema. No le gusta a los desarrolladores, a mí tampoco.

La mayoría de personas eligen un punto medio entre esas dos tipos de estrategias. Toma una visión incremental de las pruebas, comenzando con las unidades del sistema, avanza hacia las pruebas integración y culmina con las pruebas de validación.

26. Explique las pruebas de unidad

Las pruebas de unidad como se dijo anteriormente buscan verificar en la unidad más pequeña del diseño de software: El componente o módulo de software. Las rutas de control importantes se prueban para descubrir errores dentro de la frontera del módulo.

Estas pruebas se enfocan en la lógica de procesamiento interno y de las estructuras de datos dentro de las fronteras de un componente. Además se pueden realizar en paralelo con múltiples componentes

27. ¿Qué consideraciones se deben tener en las pruebas de unidad ?

Las consideraciones que se deben tener en cuenta en las pruebas de unidad son:

- La **interfaz del módulo** se prueba para garantizar que la información fluya de manera adecuada hacia y desde la unidad del software que se esta probando.

El flujo de datos a través de la interfaz de un componente se prueba antes de iniciar cualquier otra prueba. Si los datos no entran y salen de manera adecuada, todas las demás pruebas son irrelevantes.

- Las **estructuras de datos locales** se examinan para asegurar que los datos almacenados temporalmente mantienen su integridad durante todos los pasos en la ejecución de un algoritmo.
- Todas las **rutas independientes** a través de la estructura de control se ejercitan para asegurar que todos los estatutos en un módulo se ejecuten al menos una vez.

En estas pruebas, los casos de prueba deben diseñarse para descubrir errores debidos a cálculos erróneos, comparaciones incorrectas o flujo de control inadecuado.

- Las **condiciones de frontera** se prueban para asegurar que el módulo opera adecuadamente en las fronteras establecidas para limitar o restringir el procesamiento.

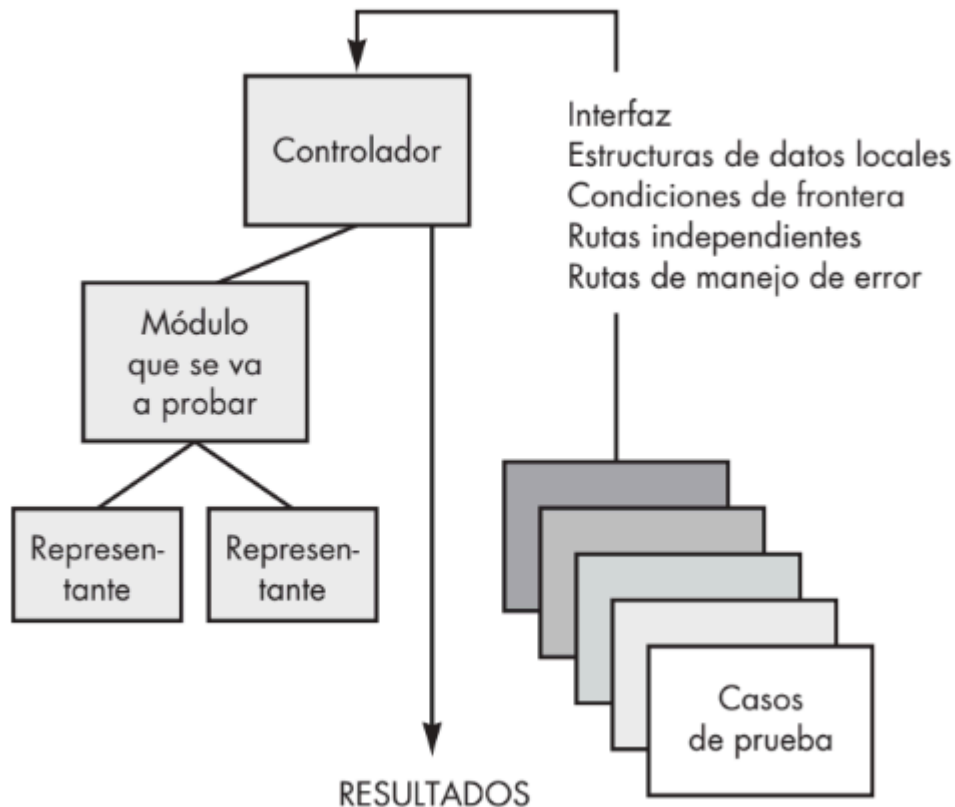
Con frecuencia los errores ocurren cuando se procesa el n -ésimo elemento de un arreglo n -dimensional, cuando se invoca la n -ésima repetición de un bucle con n pasadas, cuando se encuentra el valor máximo o mínimo permisible.

- Se ponen a prueba todas las **rutras para el manejo de errores**. Un buen diseño anticipa las condiciones de error y establece rutras de manejo de errores para enrutar o terminar limpiamente el procesamiento cuando ocurre un error. Este enfoque se llama **antierrores**.

Entre los potenciales errores que deben ponerse a prueba cuando se evalúa el manejo de errores están:

- 1) La descripción de error ininteligible.
- 2) El error indicado no corresponde con el error que se encuentra.
- 3) La condición del error causa la intervención del sistema antes de manejar el error.
- 4) El procesamiento excepción-condición es incorrecto
- 5) La descripción del error no proporciona suficiente información para auxiliar en la localización de la causa del error.

El **procedimiento de las pruebas de unidad** es que a medida que se va codificando se va probando, el diseño de las pruebas se puede realizar antes o después. Como un componente no es un programa independiente, se deben desarrollar controladores y o resguardos para cada prueba de unidad. El entorno de las pruebas de unidad se basa en esto



Un **controlador** no es nada más que un programa principal que acepta datos de un caso de prueba, pasa tales datos al componente o módulo e imprime los resultados relevantes de la prueba. (Hacer un main en java sería un controlador)

Los **representantes** o **stubs** sirven para sustituir módulos que están siendo invocados por el componente a probar. Uno de estos utiliza la interfaz de los módulos subordinados que pueden realizar una mínima manipulación de datos, imprimir verificación de entradas y regresar el control al módulo probado. (Hacer una clase pojo en java sería un representante o algo muy similar)

28. Explique las pruebas de integración

Muchos desarrolladores se preguntan para qué probar cuando se integran todos los componentes si fueron probados todos. El problema es cuando se integran todos o se conectan pueden ocurrir este tipo de errores:

- Los datos pueden perderse a través de una interfaz
- Un componente puede tener un inadvertido efecto adverso sobre otro.
- Las subfunciones, cuando se combinan, pueden no producir la función principal deseada.

- La imprecisión aceptable individualmente puede magnificarse a niveles inaceptables.
- Las estructuras de datos globales pueden presentar problemas. Lamentablemente, la lista sigue y sigue.

Las **pruebas de integración** son una técnica sistemática para construir la arquitectura del software mientras se llevan a cabo pruebas para descubrir errores asociados con la interfaz. El **objetivo principal** es tomar los componentes probados de manera individual y construir una estructura de programa que se haya dictado por diseño.

Hay una tendencia a intentar integrar todo de golpe denominado el enfoque **big bang**. Se prueba al sistema como uno solo y después dificulta la búsqueda del error ya que no está completamente aislado.

La **integración incremental** es lo contrario, se va integrando por partes sencillas o pequeñas para aislar errores, probar por completo interfaz. Para este enfoque existen diferentes estrategias como:

- **Integración descendente.** Los módulos se integran al moverse hacia abajo a través de la jerarquía de control, comenzando con el módulo de control principal (programa principal). Los módulos subordinados al módulo de control principal se incorporan en la estructura en una forma de primero en profundidad o primero en anchura.

La de **profundidad** integra cada rama según la importancia mientras que la de **anchura** integra por cada nivel.

El proceso de integración se realiza en una serie de cinco pasos:

1. El módulo de control principal se usa como un controlador de prueba y los representantes (*stubs*) se sustituyen con todos los componentes directamente subordinados al módulo de control principal.
2. Dependiendo del enfoque de integración seleccionado (es decir, primero en profundidad o anchura), los representantes subordinados se sustituyen uno a la vez con componentes reales.
3. Las pruebas se llevan a cabo conforme se integra cada componente.
4. Al completar cada conjunto de pruebas, otro representante se sustituye con el componente real.
5. Las pruebas de regresión (que se analizan más adelante en esta sección) pueden realizarse para asegurar que no se introdujeron nuevos errores.

- **Integración ascendente.** comienza la construcción y la prueba con módulos atómicos (los del último nivel). Puesto que los componentes se integran de abajo hacia arriba, la funcionalidad que proporcionan los componentes subordinados en determinado nivel siempre está disponible y se elimina la necesidad de representantes (stubs). Una estrategia de integración ascendente puede implementarse con los siguientes pasos:
 1. Los componentes en el nivel inferior se combinan en grupos (en ocasiones llamados *construcciones o builds*) que realizan una subfunción de software específica.
 2. Se escribe un *controlador* (un programa de control para pruebas) a fin de coordinar la entrada y salida de casos de prueba.
 3. Se prueba el grupo.
 4. Los controladores se remueven y los grupos se combinan moviéndolos hacia arriba en la estructura del programa.

Las **pruebas de regresión** hace referencia que a medida que se va integrando, el software va cambiando porque se establecen nuevas rutas de flujo de datos, ocurren nuevas operaciones de entrada/salida y se invoca nueva lógica de control. Por esto puede fallar funciones que antes si andaba por lo que la prueba de regresión es la nueva ejecución de algún subconjunto de pruebas que ya se realizaron a fin de asegurar que los cambios no propagaron efectos colaterales no deseados.

Las pruebas exitosas de cualquier tipo dan como resultado el descubrimiento de errores y estos deben corregirse. Siempre que se corrige el software, cambia algún aspecto de la configuración del software (el programa, su documentación o los datos que sustenta). Las pruebas de regresión ayudan a garantizar que los cambios (debidos a pruebas o por otras razones) no introducen comportamiento no planeado o errores adicionales.

Las **herramientas de captura/reproducción** permiten al ingeniero de software capturar casos de prueba y resultados para una posterior reproducción y comparación. La **suite de prueba** de regresión (el subconjunto de pruebas que se va a ejecutar) contiene tres clases diferentes de casos de prueba:

- Una muestra representativa de pruebas que ejercitará todas las funciones de software.

- Pruebas adicionales que se enfocan en las funciones del software que probablemente resulten afectadas por el cambio.
- Pruebas que se enfocan en los componentes del software que cambiaron.

Las **pruebas de humo** es un enfoque de prueba de integración que se usa cuando se desarrolla software de producto. Este abarca las siguientes actividades:

- Los componentes de software traducidos en código se integran en una **construcción**. Una construcción incluye todos los archivos de datos, bibliotecas, módulos reutilizables y componentes sometidos a ingeniería que se requieren para implementar una o más funciones del producto.
- Se diseña una serie de pruebas para exponer los errores que evitarán a la construcción realizar adecuadamente su función. La intención debe ser descubrir errores “paralizantes” que tengan la mayor probabilidad de retrasar el proyecto.
- La **construcción** se integra con otras construcciones, y todo el producto (en su forma actual) se somete a prueba de humo diariamente. El enfoque de integración puede ser descendente o ascendente.

Estas pruebas consisten en ejecutar una serie de pruebas básicas y críticas para verificar que las funciones principales de una aplicación están funcionando correctamente. Su propósito es asegurar que el software es lo suficientemente estable para proceder con pruebas más exhaustivas.

Las pruebas de humo proporciona algunos beneficios cuando se aplica sobre proyectos de software complejos y cruciales en el tiempo:

- **Se minimiza el riesgo de integración.** Puesto que las pruebas de humo se realizan diariamente, las incompatibilidades y otros errores paralizantes pueden descubrirse tempranamente, lo que reduce la probabilidad de impacto severo sobre el calendario cuando se descubren errores.
- **La calidad del producto final mejora.** Es probable que la prueba de humo descubra errores funcionales así como errores de diseño arquitectónico o en el componente debido a que el enfoque está orientado a la construcción (integración). Si tales errores se corrigen temprano, se tendrá una mejor calidad del producto.
- **El diagnóstico y la corrección de errores se simplifican.** Como todo enfoque de prueba de integración, es probable que los errores descubiertos

durante la prueba de humo se asocian con “nuevos incrementos de software”; es decir, el software que se acaba de agregar a la(s) construcción(es) es causa probable de un error recientemente descubierto.

- **El progreso es más fácil de valorar.** Con cada día que transcurre, más software se integra y se demuestra que funciona. Esto incrementa la moral del equipo y brinda a los gerentes un buen indicio de que se está progresando.

29. ¿Qué es un módulo crítico? ¿Cuáles son los productos de trabajo de las pruebas de integración?

Un **módulo crítico** tiene una o más de las siguientes características:

- 1) Aborda muchos requerimientos de software.
- 2) Tiene un alto nivel de control (reside relativamente alto en la estructura del programa).
- 3) Es complejo o propenso a errores.
- 4) Tiene requerimientos de rendimiento definidos.

Los **productos de trabajo** de las pruebas de integración se usa un plan global para la integración del software y una descripción de las pruebas específicas se documentan en una **Especificación de pruebas**. Este producto de trabajo incorpora un **plan de prueba** y un **procedimiento de prueba**, y se vuelve parte de la configuración del software.

La prueba se divide en fases y construcciones que abordan características del software funcionales y de comportamiento específicas. Las **construcciones de programas** (grupos de módulos) se crean para corresponder a cada fase.

Los siguientes criterios y pruebas correspondientes se aplican a todas las fases de prueba:

- **Integridad de interfaz.** Las interfaces internas y externas se prueban conforme cada módulo (o grupo) se incorpora en la estructura.
- **Validez funcional.** Se realizan pruebas diseñadas para descubrir errores funcionales ocultos.

- **Contenido de la información.** Se realizan pruebas diseñadas para descubrir errores ocultos asociados con las estructuras de datos locales o globales.
- **Rendimiento.** Se realizan pruebas diseñadas para verificar los límites del rendimiento establecidos durante el diseño del software.

En el **plan de pruebas** se discute:

- El calendario de integración
- El desarrollo de software sobrecargado del sistema o mejor conocidos como representantes y controladores.
- Se establecen las fechas de inicio y fin de cada fase
- Se definen “ventanas disponibles” para módulos de pruebas de unidad.
- Una breve descripción del software de sobrecarga (representantes y controladores) se concentra en las características que pueden requerir de un esfuerzo especial.
- Se describe el entorno y los recursos de la prueba.
- Configuraciones inusuales de hardware, simuladores peculiares y herramientas o técnicas de prueba especial son algunos de los muchos temas que también pueden analizarse.

En el **procedimiento de prueba** se detalla que se requiere para lograr el plan de pruebas. En este:

- Se señala el orden de la integración y las pruebas correspondientes en cada paso de ésta.
- Incluye una lista de todos los casos de prueba (anotados para referencia posterior) y los resultados esperados.

También se puede anexar un **Reporte de pruebas** en la especificación de pruebas. Se registra una historia de resultados, problemas o peculiaridades de prueba reales. La información contenida en esta sección puede ser vital durante el mantenimiento del software. También se presentan las referencias y apéndices apropiados.

30. ¿Cuáles son las estrategias para pruebas orientadas a objetos?

Las estrategias para los diferentes tipos de pruebas es encontrar el mayor número de errores con una cantidad de esfuerzo aplicado durante un tiempo realista.

Las pruebas de unidad en el contexto OO

El concepto de **unidad** cambia. La encapsulación determina la definición de clases y objetos. Esto significa que cada clase y cada instancia de una clase empaqueta los atributos (datos) y las operaciones que manipulan estos datos. En general, el foco son aquellas clases encapsuladas pero hay que tener en cuenta que los métodos de estas clases son la unidad más pequeña comprobable.

La **prueba de clase** para software OO es el equivalente de la prueba de unidad para software convencional. Para este tipo de pruebas se enfocan en las operaciones encapsuladas por la clase y el comportamiento de estado de ésta.

Prueba de integración en el contexto OO

Existen dos estrategias diferentes para la prueba de integración de los sistemas OO:

- La **prueba basada en hebra**. Está integra el conjunto de clases requeridas para responder a una entrada o evento para el sistema. Cada hebra se integra y prueba de manera individual y a su vez, se aplican las pruebas de regresión
- La **prueba basada en uso**. Se empieza a construir el sistema a medida que se van utilizando las clases independientes (Aquellas apenas usan clases servidor). Después se prueba la siguiente capa de clases, llamadas dependientes, que usan las clases independientes. Esta secuencia de probar capas de clases dependientes continúa hasta que se construye todo el sistema.

Aclaración: El uso de representantes y controladores cambia en este contexto.

Los **controladores** pueden usarse para probar operaciones en el nivel más bajo, y para la prueba de todos los grupos de clases. También puede usarse un controlador para sustituir la interfaz de usuario, de modo que las pruebas de funcionalidad del sistema puedan realizarse antes de la implementación de la interfaz.

Los **representantes** (stubs) pueden usarse en situaciones donde se requiere la colaboración entre clases pero donde una o más de las clases colaboradoras todavía no se implementan por completo.

La **prueba de grupo** es un paso en la prueba de integración del software OO. Aquí, un grupo de clases colaboradoras (determinadas al examinar el CRC y el modelo

objeto relacional) se ejercita al diseñar casos de prueba que intentan descubrir errores en las colaboraciones.

31. Explique las pruebas de validación ¿Cuáles son sus criterios?

Las **pruebas de validación** inician una vez finalizadas las pruebas de integración. En este nivel ya no importa cómo sea el software sino que las pruebas se deben enfocar en las acciones visibles para el usuario y las salidas del sistema reconocibles por el usuario.

La validación es exitosa cuando el software funciona en una forma que cumpla con las expectativas razonables del cliente. Si se desarrolló una *Especificación de requerimientos de software*, en ella se describen todos los atributos del software visibles para el usuario; contiene una sección de **Criterios de validación** que forman la base para un enfoque de pruebas de validación.

Como se dijo la validación se logra a través de una serie de pruebas que demuestran conformidad con los requerimientos.

Un **plan de pruebas** subraya las clases de pruebas que se van a realizar y un **procedimiento de prueba** define casos de prueba específicos que se diseñan para garantizar los requerimientos.

Una vez finalizada las pruebas pueden ocurrir dos situaciones:

1. La característica de función o rendimiento se conforma de acuerdo con las especificaciones y se acepta
2. Se descubre una desviación de la especificación y se crea una lista de deficiencias.

En el segundo caso se debe renegociar con los clientes para ver cómo se resolverá estas deficiencias

Cuando se construye software a la medida para un cliente, se realizan una serie de **pruebas de aceptación** a fin de permitir al cliente validar todos los requerimientos. Este tipo de pruebas las realiza el usuario y no el desarrollador, en sí puede variar en informal o muy formal y pueden tardar semanas o meses.

Cuando se hace un software para muchos clientes, no es práctico hacer pruebas de aceptación formales por lo que se recurren a las pruebas alfa y beta

32. ¿Qué son las pruebas alfa y beta?

La **prueba alfa** se lleva a cabo en el sitio del desarrollador por un grupo representativo de usuarios finales. El software se usa en un escenario natural con el desarrollador “mirando sobre el hombro” de los usuarios y registrando los errores y problemas de uso. Las pruebas alfa se realizan en un **ambiente controlado**.

La **prueba beta** se realiza en uno o más sitios del usuario final por lo general el desarrollador no está presente. Por lo tanto, la prueba beta es una aplicación “en vivo” del software en un ambiente que no puede controlar el desarrollador. El cliente registra todos los problemas (reales o imaginarios) que se encuentran durante la prueba beta y los reporta al desarrollador periódicamente. Como resultado de los problemas reportados durante las pruebas beta, es posible hacer modificaciones y luego preparar la liberación del producto de software a toda la base de clientes.

En ocasiones se realiza una variación de la prueba beta, llamada **prueba de aceptación del cliente**, cuando el software se entrega a un cliente bajo contrato. El cliente realiza una serie de pruebas específicas con la intención de descubrir errores antes de aceptar el software del desarrollador. (En ZimaTech lo viví esto uwu)

33. Explique las pruebas de sistemas

La **prueba del sistema** es una serie de diferentes pruebas cuyo propósito principal es ejercitar por completo el sistema basado en computadora, incluyendo no solo el software, sino también otros elementos del sistema como el hardware, las personas y la información.

Aunque cada prueba tenga un propósito diferente, todo él funciona para verificar que los elementos del sistema se hayan integrado de manera adecuada y que se realicen las funciones asignadas.

Los tipos de pruebas del sistema que valen la pena para los sistemas basados en software son:

- **Pruebas de recuperación.**

Muchos sistemas basados en computadora deben recuperarse de fallas y reanudar el procesamiento con poco o ningún tiempo de inactividad. Hay veces que el sistema debe ser tolerable a fallas, es decir, las fallas del procesamiento no deben causar el cese del funcionamiento del sistema.

La **recuperación** es una prueba de sistema que fuerza al software a fallar en varias formas y que verifica que la recuperación se realice de manera adecuada. Si es automática se evalúa el reinicio, los mecanismos de puntos de verificación, la recuperación de los datos y la reanudación para correcciones. Si no lo es se mide el tiempo medio de reparación (TMR) para determinar si está dentro de los límites aceptables

- **Pruebas de seguridad.**

Este tipo de pruebas intenta verificar que los mecanismos de protección que se construyen en un sistema en realidad lo protegerán de cualquier penetración impropia.

Con los suficientes tiempo y recursos, las buenas pruebas de seguridad a final de cuentas penetran en el sistema. El papel del diseñador de sistemas es hacer que el costo de la penetración sea mayor que el valor de la información que se obtendrá.

La **penetración** abarca un amplio rango de actividades: Hackers que intentan penetrar en los sistemas por deporte, empleados resentidos que intentan penetrar por venganza, individuos deshonestos que intentan penetrar para obtener ganancia personal ilícita.

- **Pruebas de esfuerzo.**

La prueba de esfuerzo ejecuta un sistema en forma que demanda recursos en cantidad, frecuencia o volumen anormales. Por ejemplo, se pueden diseñar pruebas especiales que generen diez interrupciones por segundo, cuando una o dos es la tasa promedio, aumentarse las tasas de entrada de datos en un orden de magnitud para determinar cómo responderán las funciones de entrada, etc. En esencia, la persona que prueba intenta romper el programa.

Existe una variante llamada **pruebas de sensibilidad** que intenta descubrir combinaciones de datos dentro de clases de entrada válidas que puedan

causar inestabilidad o procesamiento inadecuado. Normalmente se ven en algoritmos matemáticos y hace referencia a que con un rango pequeño de datos que estén en las fronteras de los datos válidos pueden causar procesamiento extremo, e incluso erróneo, o una degradación del rendimiento.

- **Pruebas de rendimiento.**

La prueba de rendimiento se diseña para poner a prueba el rendimiento del software en tiempo de corrida, dentro del contexto de un sistema integrado. La prueba del rendimiento ocurre a lo largo de todos los pasos del proceso de prueba (de unidad, de integración, de validación, de sistema) pero solo se puede determinar el rendimiento cuando todos los elementos del sistema están completamente integrados.

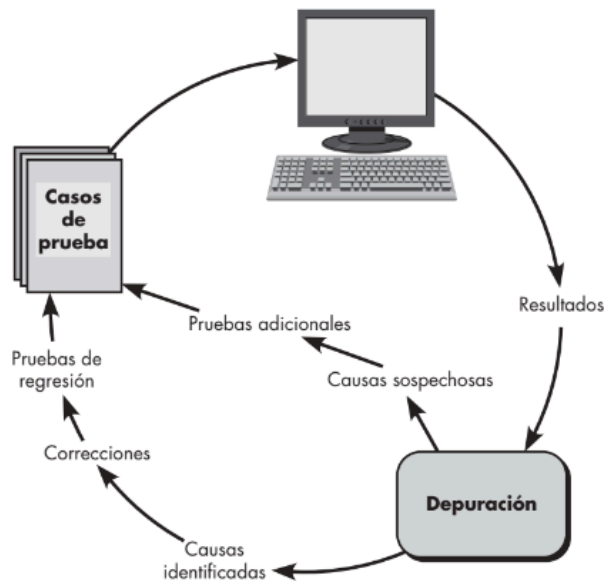
- **Pruebas de despliegue.**

La prueba de despliegue, en ocasiones llamada prueba de configuración, ejercita el software en cada entorno en el que debe operar. Además, examina todos los procedimientos de instalación y el software de instalación especializado (por ejemplo, “instaladores”) que usarán los clientes, así como toda la documentación que se usará para introducir el software a los usuarios finales.

34. Explique la depuración y sus estrategias

La **depuración** ocurre cuando un caso de prueba descubre un error, y es el proceso que da como resultado la remoción del error. Aunque la depuración puede y debe ser un proceso ordenado, todavía en mucho es un arte.

El proceso de depuración es:



Este proceso parte como consecuencia de una prueba exitosa. Los resultados de las pruebas se valoran y se encuentra la falta de correspondencia entre el rendimiento esperado y el real.

La depuración tiene dos posibilidades: Encontrar el error y corregirlo o no encontrar el error.

¿Por qué es tan difícil la depuración?

Por estas razones:

- El síntoma y la causa pueden ser geográficamente remotos. Es decir, el síntoma puede aparecer en una parte de un programa, mientras que la causa en realidad puede ubicarse en un sitio que esté alejado.
- El síntoma puede desaparecer (temporalmente) cuando se corrige otro error.
- El síntoma en realidad puede no ser causado por errores (por ejemplo, imprecisiones de redondeo).
- El síntoma puede ser causado por un error humano que no se rastrea con facilidad.
- El síntoma puede ser resultado de problemas de temporización más que de problemas de procesamiento.
- Puede ser difícil reproducir con precisión las condiciones de entrada (por ejemplo, una aplicación en tiempo real en la que el orden de la entrada esté indeterminado).

- El síntoma puede ser intermitente, particularmente común en sistemas embebidos que acoplan hardware y software de manera inextricable.
- El síntoma puede deberse a causas que se distribuyen a través de algunas tareas que corren en diferentes procesadores.

Para la depuración en el libro de Roger Pressman se proponen 3 estrategias:

- **Fuerza bruta.** Se utiliza si no funcionan las otras dos y sigue la filosofía de dejar que la computadora encuentre el error por lo que empiezan a tomar copias de la memoria, se invocan rastreos en tiempo de corrida y se carga con enunciados de salida.
- **Vuelta atrás (del inglés backtracking).** Es un enfoque que se usa en programas pequeños. Es ubicar en donde comenzó el síntoma y se empieza a rastrear hacia atrás hasta encontrar las causas. Mientras mas grande el software peor
- **Eliminación de causas.** Utiliza inducción o deducción, e introduce el concepto de partición binaria. Los datos utilizados cuando ocurrió el error se organizan para aislar las causas potenciales. Se plantea una hipótesis de causa y los datos se utilizan para probar o refutar la hipótesis. Sino se plantea una lista de posibles causas y se realizan pruebas para eliminar cada una.

Para poder corregir el error una vez se encuentra se debe responder a las siguientes preguntas:

- ¿La causa del error se reproduce en otra parte del programa? En muchas situaciones, un defecto de programa es causado por un patrón de lógica erróneo que puede reproducirse en alguna otra parte. La consideración explícita del patrón lógico puede resultar en el descubrimiento de otros errores.
- ¿Qué “siguiente error” puede introducirse con la corrección que está a punto de realizar? Antes de hacer la corrección, debe evaluarse el código fuente (o, mejor, el diseño) para valorar el acoplamiento de las estructuras lógica y de datos. Si la corrección se realizará en una sección altamente acoplada del programa, debe tenerse especial cuidado cuando se realice algún cambio.

- ¿Qué debió hacerse para evitar este error desde el principio? Esta pregunta es el primer paso hacia el establecimiento de un enfoque de aseguramiento de calidad estadística del software. Si se corrigen tanto el proceso como el producto, el error se removerá del programa actual y podrá eliminarse de todos los programas futuros.

Unidad 5 - Pruebas orientadas a objetos

Libros:

Cap 19 -  Roger S Pressman - Ingeniería del software _ un enfoque práctico (7...

 Metodo-Rebanadas.pdf

Preguntontas

1. ¿Qué se necesita para probar los sistemas OO?

Para probar adecuadamente los sistemas OO, deben realizarse tres cosas:

- Ampliar la definición de prueba para incluir las técnicas de descubrimiento de error aplicadas al análisis orientado a objetos y a modelos de diseño.
- Cambiar significativamente la estrategia para prueba de unidad e integración.
- Explicar las características únicas del software OO mediante el diseño de casos de prueba.

2. ¿Qué significa probar en OO?

Los sistemas OO empiezan con representaciones relativamente informales de los requisitos de sistema y evolucionan a un modelo más detallado de clases, relaciones de clases, diseño y asignación de sistema, y diseño de objetos. En cada etapa, el modelo puede probarse con la intención de descubrir errores para evitar su propagación.

En las etapas de análisis y diseño OO proporcionan información sobre la estructura y comportamiento del sistema por lo que debe estar sujeta a una rigurosa revisión. En este contexto, el término prueba incorpora revisiones técnicas.

Todos los modelos OO deben probarse en relación con su exactitud, completitud y consistencia dentro del contexto de la sintaxis, la semántica y la pragmática del modelo

3. ¿Cuáles son los modelos AOO y DOO?

El análisis OO y el diseño OO no se puede probar como tal sino que se debe realizar una revisión técnica para examinar su exactitud y consistencia.

La sintaxis y notación se probará según el método que se use en estas etapas por lo que se deben inspeccionar los modelos para que cumplan las convenciones adecuadas

La exactitud semántica se puede valorar según la conformidad del modelo con el dominio de problemas del mundo real. Si este modelo refleja el mundo real entonces es semánticamente correcto.

4. ¿Qué es una prueba de unidad en OO?

La encapsulación impulsa la definición de clases y objetos. Esto significa que cada clase y objeto encapsulan los atributos (datos) y las operaciones (también conocidas como métodos o servicios) que manipulan dichos datos. **La unidad comprobable más pequeña es la clase encapsulada.**

Puesto que una clase puede contener algunas operaciones diferentes y una operación particular puede existir como parte de un número de clases diferentes, el significado de prueba de unidad cambia dramáticamente. Por ejemplo una Clase A utiliza un método `funcionMisteriosa()` y si la Clase B hereda de A, no se puede probar solo la `funcionMisteriosa` ya que en B utilizará los atributos de B y no los de A entonces no se puede garantizar una prueba de unidad convencional.

La prueba de clase para el software OO se activa mediante las operaciones encapsuladas por la clase y por el comportamiento de estado de la misma.

5. ¿Qué es una prueba de integración en OO?

Existen dos estrategias diferentes para la prueba de integración de los sistemas OO:

- La **prueba basada en hebra**. Está integrada el conjunto de clases requeridas para responder a una entrada o evento para el sistema. Cada hebra se integra y prueba de manera individual y a su vez, se aplican las pruebas de regresión.
- La **prueba basada en uso**. Se empieza a construir el sistema a medida que se van utilizando las clases independientes (Aquellas apenas usan clases servidor). Después se prueba la siguiente capa de clases, llamadas dependientes, que usan las clases independientes. Esta secuencia de probar

capas de clases dependientes continúa hasta que se construye todo el sistema.

6. ¿Qué es una prueba de validación en OO?

En el nivel de validación o de sistema, desaparecen los detalles de las conexiones de clase.

Los métodos convencionales de prueba de caja negra pueden usarse para activar pruebas de validación. Además, puede elegirse derivar casos de prueba del modelo de comportamiento del objeto y de un diagrama de flujo de evento creado como parte del AOO.

7. ¿Cuáles son los métodos de pruebas orientados a objetos?

Los métodos de diseño de casos de prueba para el software orientado a objetos siguen evolucionando. Sin embargo, se sugiere un enfoque global en el diseño de casos de prueba OO:

- Cada caso de prueba debe identificarse de manera única y explícita asociado con la clase que se va a probar.
- Debe establecerse el propósito de la prueba.
- Debe desarrollarse una lista de pasos de prueba para cada una de ellas, que debe contener:
 - Una lista de estados especificados para la clase que se probará.
 - Una lista de mensajes y operaciones que se ejercitarán como consecuencia de la prueba.
 - Una lista de excepciones que pueden ocurrir conforme se prueba la clase.
 - Una lista de condiciones externas (es decir, con la finalidad de realizar adecuadamente las pruebas, cambios en el entorno externo al software que debe existir)
 - Información complementaria que ayudará a comprender o a implementar la prueba

A diferencia de las pruebas tradicionales, las pruebas orientadas a objetos se enfocan en el diseño de secuencias apropiadas de operaciones para ejercitar los estados de una clase.

Los métodos que se utilizan son:

- **Prueba basada en fallo.**

Se deben diseñar pruebas que tengan altas probabilidades de descubrir fallos plausibles, es decir, aspectos de la implementación del sistema que pueden resultar en defectos.

La prueba de integración busca fallos plausibles en los llamados de operación y en las conexiones de mensaje. En este contexto se encuentran tres tipos de fallos: Resultado inesperado, uso de operación/mensaje equivocado e invocación incorrecta.

Los “comportamientos” de un objeto están definidos por los valores que le son asignados a sus atributos. Las pruebas deben ejercer los atributos para determinar si los valores adecuados ocurren para los distintos tipos de comportamiento de los objetos. Además el foco de la prueba de integración es determinar si existen errores en el código que llama, no en el código llamado.

- **Diseño de pruebas basadas en escenarios.**

Las pruebas basadas en fallo pierden dos tipos principales de errores:

- 1) Especificaciones incorrectas.
- 2) Interacciones entre subsistemas.

Cuando ocurren errores asociados con una especificación incorrecta, el producto no hace lo que el cliente quiere. Puede hacer lo correcto u omitir funcionalidad importante. Los errores asociados con la interacción de subsistemas ocurren cuando el comportamiento de un subsistema crea circunstancias (por ejemplo, eventos, flujo de datos) que hacen que otro subsistema falle.

Este tipo de prueba se concentra en lo que hace el usuario, es decir, capturar las tareas (por medio de casos de uso) que el usuario tiene que realizar y luego aplicar éstas y sus variantes como pruebas.

Los **escenarios** descubren errores de interacción. Pero, para lograr esto, los casos de prueba deben ser más complejos y más realistas que las pruebas basadas en fallo. La prueba basada en escenario tiende a ejercitar múltiples subsistemas en una sola prueba.

- **Pruebas de las estructuras superficiales y profundas.**

Este tipo de pruebas trae consigo el concepto de **estructura superficial** que hace referencia a la estructura observable externamente de un programa OO (la que es inmediatamente obvia para cada usuario final). Pero, cualquiera que sea la interfaz, las pruebas se basan todavía en tareas de usuario y se requiere comprensión y observación de cómo los usuarios interactúan con el sistema.

Cuando se habla de **estructuras profundas** hace referencia a los detalles técnicos internos del programa, como dependencias, comportamientos y mecanismos de comunicación entre objetos. Utilizan los modelos de requerimientos y diseño (como diagramas de colaboración UML) para verificar interacciones no visibles externamente.

Aclaración: Si una clase Base tiene los métodos llamados inherited() y redefined(). Una clase llamada Derivada hereda Base, Derivada redefine el método redefined() pero no hace nada con el método Inherited. No se debe probar dos veces el método inherited a menos que este llame a la función redefined.

8. ¿Cuáles son las implicaciones del diseño de casos de prueba OO?

La **encapsulación** puede crear un obstáculo menor cuando se prueba porque para las pruebas se necesita reportar el estado concreto y abstracto de un objeto y la encapsulación complica la obtención de este tipo de información.

La **herencia** también presenta un reto ya que para cada nueva clase es un nuevo contexto por lo que requiere de un nuevo examen, si se usa herencia múltiples los contextos se disparan pero si se usa en el mismo contexto que la superclase se pueden utilizar los mismo casos de prueba.

9. ¿Cuáles son los métodos de pruebas aplicados en nivel unidad?

La prueba en lo pequeño o de unidad se enfoca en una sola clase y en los métodos encapsulados.

Los métodos que se utilizan en este tipo de pruebas son:

- **Prueba aleatoria para clases OO.** Se consideran las restricciones pertinentes al dominio del problema y se prueban métodos aleatorios. Por ejemplo en una cuenta tenes el método iniciar sesión y cerrar sesión junto a 30 métodos más y una combinación aleatoria entre iniciar sesión y cerrar sesión puede ocurrir.
- **Prueba de partición en el nivel de clase.** Las pruebas de partición reducen el número de casos de pruebas para ejercitar las clases. Es similar a las pruebas de partición equivalente. Las entradas y salidas se categorizan y los casos de prueba se diseñan para ejercitar cada categoría.

La **partición con base en estado** categoriza las operaciones de clase a partir de su capacidad para cambiar el estado de la clase (Cambiar el valor de sus atributos). A partir de ahí, se debe probar esa separación.

La **partición con base en atributos** categoriza las operaciones de clase con base en los atributos que usan. Se separa según qué atributos se usan en el método, es decir, se puede separar por cada atributo la partición que usa el valor del atributo, la que lo modifica y los que no modifica ni usa.

La **partición basada en categoría** jerarquiza las operaciones de clase con base en la función genérica que cada una realiza. Le pone una categoría a cada método y se prueba desde ahí.

10. ¿Cómo es el diseño de pruebas interclase?

El diseño de casos de prueba se vuelve más complicado conforme comienza la integración del sistema orientado a objetos. En esta etapa debe comenzar la prueba de las colaboraciones entre clases. Para este tipo de prueba pueden aplicarse los métodos aleatorios y de partición, así como pruebas basadas en escenario y pruebas de comportamiento.

Para las pruebas de método aleatorio se plantea las siguientes sugerencias:

- Para cada clase cliente, use la lista de operaciones de clase a fin de generar una serie de secuencias de prueba aleatorias. Las operaciones enviarán mensajes a otras clases servidor.
- Para cada mensaje generado, determine la clase colaborador y la correspondiente operación en el objeto servidor.
- Para cada operación en el objeto servidor (invocado por los mensajes enviados desde el objeto cliente), determine los mensajes que transmite.
- Para cada uno de los mensajes, determine el siguiente nivel de operaciones que se invocan e incorpore esto en la secuencia de prueba.

El enfoque de **prueba de partición** de clase múltiple es similar al que se usó para la prueba de partición de clases individuales. Una sola clase se divide. Sin embargo, la secuencia de prueba se expande para incluir aquellas operaciones que se invocan mediante mensajes a clases que colaboran.

El **diagrama de estado** para una clase puede usarse a fin de ayudar a derivar una secuencia de pruebas que ejercitará el comportamiento dinámico de la clase (y de aquellas clases que colaboran con ella). Las pruebas que se van a diseñar deben lograr cobertura de todos los estados.

11. ¿Qué es una rebanada?

Para entender el concepto de rebanada hay que plantear que es un objeto. Cuando se habla de objetos se considera una abstracción definida por un nombre, un estado y un conjunto de operaciones o métodos que definen su comportamiento. El estado usualmente es dinámico, afectado por la realización de los diversos métodos.

Si se considera el estado de un objeto con detenimiento, puede observarse que no todos los métodos interactúan con todos los atributos sino con un subconjunto de estos. Visto de otra manera, si se fija en un solo atributo se obtendrá un subconjunto de métodos que solo se relacionan con ese atributo. Esto es similar a cortar la clase en **rebanadas**. En las rebanadas solo debe existir un único atributo y los métodos que se relacionan con ese atributo.

El método de rebanadas consiste de dos grandes pasos:

- 1) Generar las rebanadas correspondientes a una clase
- 2) Para cada rebanada generar secuencias de métodos

Para generar las rebanadas se emplean dos estructuras de datos auxiliares: un Grafo de Llamadas Mejorado (GLLM) y una Matriz de Uso Mínima (MUM).

12. ¿Qué es un grafo de llamada?

Un **Grafo de Llamadas Mejorado** (GLLM) de una clase es un grafo constituido por uno o más árboles, donde cada árbol tiene como raíz un método de la clase y de ella parten ramas a los atributos a que hace referencia y a los métodos de la misma clase que invoca. Los métodos se representan como nodos circulares y los atributos como nodos cuadrados.

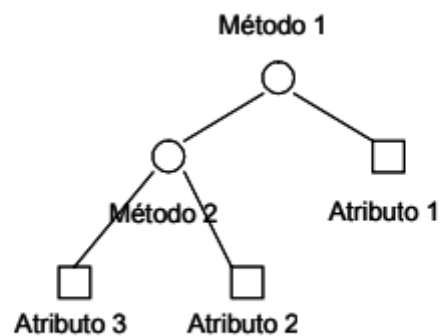


Figura 1 Grafo de Llamadas mejorado

13. ¿Qué es la matriz de uso mínima?

La **Matriz de Uso Mínima** es una matriz que tiene una fila por cada atributo de una clase y una columna por cada método de la misma, incluyendo los constructores.

La celda tendrá un contenido que depende de la relación entre el método y el atributo, como sigue:

- t → si el método transforma o modifica el valor del atributo.
- r → si el método reporta o informa el valor del atributo.
- o → si el método utiliza el valor del atributo en alguna operación, sin cambiarlo.
- Vacío → si no hay relación entre ellos.

14. ¿Cómo se hace una secuencia de métodos?

Una vez que se cuenta con la MUM, se debe hacer lo siguiente:

- Se categoriza cada método como: constructores, reporteros, transformadores y otros. La categoría transformadores incluye los métodos que realizan

modificaciones sobre los atributos y también los que operan con sus valores, es decir los que se marcaron con “t” y con “o”. En otros se dejan los que no caen en ningún caso anterior, que se supone son excepciones, como métodos que reportan errores y mandan avisos auxiliares, sin relación con el estado del objeto.

- Se prueba a los métodos reporteros. En este paso sólo interesa que reporten adecuadamente el valor del atributo que les corresponde. Si existe un atributo sin reportero, Si el atributo es privado, será imposible probar adecuadamente la clase, a menos que se modifique el código. Si el atributo es protegido (friend), se puede generar una clase derivada de la clase que se está probando, agregando los reporteros necesarios, lo cual no altera la clase original. Pueden generar secuencias de reporteros al azar.
- Se prueban los diversos constructores, con diferentes valores y se verifican éstos usando los reporteros. Debe cuidarse que los reporteros cubren todas las rebanadas.
- Se prueban los transformadores como sigue:
 - Se elige un atributo y los transformadores que corresponden a su rebanada y el reportero del atributo.
 - Se forman las diversas permutaciones de los diferentes transformadores, colocando siempre el reportero al final
 - Cada una de las secuencias anteriores será una secuencia de prueba; para cada una de ellas deberán considerarse varios casos de prueba, correspondientes a los diferentes tipos de valores que pueden tomar sus parámetros.
- Una vez probados los métodos anteriores, sólo quedan los métodos que no caen en ninguna de las categorías anteriores, pero que resultan más sencillos al no afectar a ningún atributo.

15. ¿Cómo se generan los casos de prueba?

La generación de los casos de pruebas se debe acompañar con el método de particiones ya que si existe una operación que reciba parámetros el valor puede tener dos casos probables que funcione correctamente y que salte un error por lo que no funciona.

16. ¿Cómo funciona este método con clases derivadas?

En una clase derivada se presentan cuatro posibles escenarios:

- Métodos nuevos o redefinidos que afectan atributos de la clase base
- Métodos de la clase base que afectan atributos de la clase base
- Métodos de la clase base que afectan atributos nuevos
- Métodos nuevos o redefinidos que afectan atributos nuevos

De los cuatro casos sólo el segundo se puede considerar libre de nuevas pruebas.

En si se utiliza los mismos métodos pero con un grado de similitud:

- Construir el Grafo de Llamadas Mejorado (GLLM)
- Construir la matriz MUM modificada
- Probar las rebanadas correspondientes a atributos nuevos
- Probar las rebanadas correspondientes a atributos originales que hayan sufrido cambios

1.6.1 Grafo de llamadas mejorado

En forma similar al método básico, se construye el GLLM, con los siguientes cambios: cuando se invoca a un atributo de la clase original, se marcará con un cuadrado doble y cuando se invoque a un método de la clase base, se marcará con un círculo doble. En la Figura 10 se muestra el GLLM para CuentaLimitada.

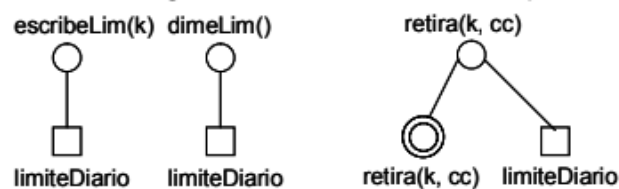


Figura 10 GLLM para la clase CuentaLimitada

Puede observarse en la Figura 10 que el método retira (nuevo) hace una llamada al método antiguo (con doble círculo).

La MUM

Tabla 4 MUM modificada

	Métodos de clase base	Métodos nuevos
Atributos de clase base	II	I
Atributos nuevos	III	IV

El llenado de la MUM debe realizarse como sigue:

- a) primero se llena el cuadrante II con los datos de la clase base;
- b) después se llena el cuadrante IV, que es totalmente nuevo
- c) después se llena el cuadrante III que probablemente esté vacío
- d) finalmente se llena el cuadrante I

1.6.3 Generación de rebanadas y secuencias

Teniendo la MUM de la clase derivada, se procede a generar las rebanadas de los atributos agregados y a partir de ellos las secuencias de prueba. Para esto se usan los cuadrantes inferiores (III y IV).

Una vez terminado el paso anterior, se procede a analizar las rebanadas de la clase base y observar si hubo cambios. Si los hay, entonces se deben generar las secuencias de prueba igual que antes. Para las rebanadas que no sufrieron cambio no es necesario hacer nada. En esta etapa se usan los cuadrantes superiores (I y II).

Unidad 6 - Pruebas de integración de sistemas y validación

Libros:

Cap 18 -  Roger S Pressman-Ingenieria.del.Software-5ta edicion.pdf

Preguntontas:

No se necesita realizar ninguna preguntontas porque ya está explicado en la unidad 4.

Unidad 7 - Ciclo de vida del Testing

Libros:

Cap 1 Secc 4 ■ Foundations of software testing - ISTQB Certification.pdf

Preguntontas

1. ¿Cómo se separa el ciclo de vida del testing? ¿Son secuenciales?

El ciclo de vida del testing se separa de la siguiente manera:

- Planificación y control.
- Análisis y diseño.
- Implementación y ejecución.
- Evaluación de los criterios de salida y reporte.
- Actividades de cierre de prueba.

Estas etapas suelen ser secuenciales pero pueden sobreponerse, realizarlas de manera concurrente e incluso repetirse. Este tipo de etapas están pensadas para pruebas dinámicas pero los encabezados principales del proceso también pueden aplicarse a las revisiones. Por ejemplo, necesitamos planificar y preparar revisiones, llevar a cabo las revisiones y evaluar los resultados de las revisiones. Para algunas revisiones, como las inspecciones, tendremos criterios de salida y pasaremos por actividades de cierre.

2. ¿Qué se hace en la planificación y control de pruebas?

Durante la planificación de pruebas, nos aseguramos de comprender los objetivos y metas de los stakeholders y los riesgos que las pruebas pretenden abordar. Esto nos otorgará la misión de las pruebas o asignación de pruebas.

Gracias a esto se podrá utilizar un enfoque de pruebas y un plan de pruebas, incluida la especificación de actividades de prueba.

Para ayudarnos, podemos tener políticas de prueba de la organización o del programa y una estrategia de prueba. La **política de prueba** da reglas para las pruebas, por ejemplo, 'siempre revisamos los documentos de diseño'. La **estrategia de prueba** es el enfoque general de alto nivel, por ejemplo, las pruebas del sistema son llevadas a cabo por un equipo independiente que informa al gerente de calidad del programa.

La planificación de pruebas tiene las siguientes tareas principales, dadas aproximadamente en orden, que nos ayudan a construir un plan de prueba:

- **Determinar el alcance y los riesgos e identificar los objetivos de las pruebas:** Consideramos qué software, componentes, sistemas u otros productos están dentro del alcance de las pruebas. También los diferentes tipos de riesgos que necesitan ser abordados. Para finalizar, se debe considerar qué tipo de pruebas estamos haciendo si es para descubrir defectos, para mostrar que el software cumple con los requisitos, para demostrar que el sistema es adecuado para su propósito o para medir las cualidades y atributos del software.
- **Determinar el enfoque de la prueba (técnicas, elementos de prueba, cobertura, identificación e interacción con los equipos involucrados en la prueba, testware):** Consideramos cómo llevaremos a cabo la prueba, las técnicas a usar, qué necesita ser probado y en qué medida (es decir, qué extensión de la cobertura). Veremos quién necesita involucrarse y cuándo (esto podría incluir desarrolladores, usuarios, equipos de infraestructura de TI); decidiremos qué vamos a producir como parte de las pruebas (por ejemplo, testware como procedimientos de prueba y datos de prueba). Esto estará relacionado con los requisitos de la estrategia de prueba.

Aclaración: El término "testware" se refiere a todos los artefactos que se producen durante el proceso de prueba de software

- **Implementar la política de prueba y/o la estrategia de prueba:** Mencionamos que puede haber una política y estrategia de prueba de la organización o del programa. Si este es el caso, durante nuestra planificación debemos asegurarnos de que lo que planeamos hacer se adhiera a la política y la estrategia o debemos haber acordado con los interesados y documentado una buena razón para desviarnos de ella.
- **Determinar los recursos de prueba necesarios (por ejemplo, personas, entorno de prueba, PCs):** A partir de la planificación que ya hemos hecho, ahora podemos entrar en detalle. Decidimos la composición de nuestro equipo y también configuramos todo el hardware y software de apoyo que necesitamos para el entorno de prueba.
- **Programar las tareas de análisis y diseño de prueba, implementación, ejecución y evaluación:** Necesitaremos un cronograma de todas las tareas

y actividades, para poder hacer un seguimiento de ellas y asegurarnos de que podemos completar las pruebas a tiempo.

- **Determinar los criterios de salida:** Necesitamos establecer criterios como los criterios de cobertura (por ejemplo, el porcentaje de declaraciones en el software que deben ejecutarse durante las pruebas) que nos ayudarán a seguir si estamos completando las actividades de prueba correctamente. Nos mostrarán qué tareas y verificaciones debemos completar para un nivel particular de prueba antes de poder decir que las pruebas han terminado.

La gestión de cualquier actividad no se detiene con planificarla. Necesitamos **controlar y medir el progreso contra el plan**. Por lo tanto, el **control de pruebas** es una actividad continua. Necesitamos comparar el progreso real con el progreso planificado, y reportar al gerente de proyecto y al cliente sobre el estado actual de las pruebas, incluidos cualquier cambio o desviación del plan. Necesitaremos tomar acciones donde sea necesario para cumplir con los objetivos del proyecto. Tales acciones pueden implicar cambiar nuestro plan original.

Cuando diferentes grupos realizan diferentes actividades de revisión y prueba dentro del proyecto, la planificación y el control deben ocurrir dentro de cada uno de esos grupos pero también debe ocurrir por encima de los grupos para coordinar entre ellos, permitiendo transiciones fluidas entre cada etapa de la prueba. La planificación de pruebas tiene en cuenta la retroalimentación de las actividades de monitoreo y control que se llevan a cabo a lo largo del proyecto.

El **control de pruebas** tiene las siguientes tareas principales:

- **Medir y analizar los resultados de las revisiones y pruebas:** Necesitamos saber cuántas revisiones y pruebas hemos hecho. Necesitamos rastrear cuántas pruebas han pasado y cuántas han fallado, junto con el número, tipo e importancia de los defectos reportados.
- **Monitorear y documentar el progreso, la cobertura de pruebas y los criterios de salida:** Es importante que informemos al equipo del proyecto cuánto se ha probado, cuáles son los resultados y qué conclusiones y evaluación de riesgos hemos hecho. Debemos hacer que el resultado de la prueba sea visible y útil para todo el equipo.
- **Proporcionar información sobre las pruebas:** Debemos esperar hacer informes regulares y excepcionales al gerente de proyecto, patrocinador del

proyecto, cliente y otros interesados clave para ayudarles a tomar decisiones informadas sobre el estado del proyecto. También debemos usar la información que tenemos para analizar las pruebas en sí.

- **Iniciar acciones correctivas:** Por ejemplo, ajustar los criterios de salida para defectos corregidos, pedir más esfuerzo en depuración o priorizar defectos para corregir bloqueadores de prueba.
- **Tomar decisiones:** Basándonos en las medidas y la información recopilada durante las pruebas y cualquier cambio en los riesgos del negocio y del proyecto o nuestra mayor comprensión de los riesgos técnicos y del producto, tomaremos decisiones o permitiremos que otros tomen decisiones: continuar con las pruebas, detener las pruebas, liberar el software o retenerlo para un trabajo posterior, por ejemplo.

3. ¿Qué se hace en el análisis y diseño de pruebas?

El **análisis y diseño de pruebas** es la actividad donde los objetivos generales de prueba se transforman en condiciones de prueba tangibles y diseños de prueba. Durante el análisis y diseño de pruebas, tomamos los objetivos generales de prueba identificados durante la planificación y construimos diseños de prueba y procedimientos de prueba (guiones). El análisis y diseño de pruebas tiene las siguientes tareas principales, en aproximadamente el siguiente orden:

- **Revisar la base de prueba** (como el análisis de riesgos del producto, requisitos, arquitectura, especificaciones de diseño e interfaces), **examinando las especificaciones del software que estamos probando.** Usamos la base de prueba para ayudarnos a construir nuestras pruebas. Podemos comenzar a diseñar ciertos tipos de pruebas (llamadas pruebas de caja negra) antes de que exista el código, ya que podemos usar los documentos de la base de prueba para entender qué debería hacer el sistema una vez construido. A medida que estudiamos la base de prueba, a menudo identificamos brechas y ambigüedades en las especificaciones, porque estamos tratando de identificar cómo construir pruebas para cubrir estas especificaciones.

Aclaración: El término "base de prueba" se refiere al conjunto de documentos y fuentes de información en los que se basan los casos de prueba y los procesos de

prueba en general. Estos documentos proporcionan los requisitos, especificaciones, y otros criterios necesarios para diseñar y ejecutar pruebas de software

- **Identificar condiciones de prueba basadas en el análisis de los elementos de prueba, sus especificaciones y lo que sabemos sobre su comportamiento y estructura.** Esto nos da una lista de alto nivel de lo que nos interesa probar. Por ejemplo, para manejar, el examinador podría tener una lista de condiciones de prueba que incluyan 'comportamiento en intersecciones de caminos', 'uso de indicadores', 'capacidad para maniobrar el coche', y así sucesivamente. En las pruebas, utilizamos las técnicas de prueba para ayudarnos a definir las condiciones de prueba. A partir de esto, podemos empezar a identificar el tipo de datos de prueba genéricos que podríamos necesitar.
- **Diseñar las pruebas**, utilizando técnicas para ayudar a seleccionar pruebas representativas que se relacionen con aspectos particulares del software que conllevan riesgos o que son de particular interés, basándonos en las condiciones de prueba y entrando en más detalle. Por ejemplo, el examinador de manejo podría mirar una lista de condiciones de prueba y decidir que las intersecciones necesitan incluir intersecciones en T, cruces y así sucesivamente. En las pruebas, definiremos el caso de prueba y los procedimientos de prueba.
- **Evaluar la capacidad de prueba de los requisitos y del sistema.** Los requisitos pueden estar escritos de una manera que permita a un evaluador diseñar pruebas; por ejemplo, si el rendimiento del software es importante, eso debería especificarse de una manera que pueda ser probada. Si los requisitos solo dicen 'el software necesita responder lo suficientemente rápido', eso no es comprobable, porque 'lo suficientemente rápido' puede significar cosas diferentes para diferentes personas. Un requisito más comprobable sería 'el software necesita responder en 5 segundos con 20 personas conectadas'. La **capacidad de prueba** del sistema depende de aspectos tales como si es posible configurar el sistema en un entorno que coincida con el entorno operativo y si todas las formas en que el sistema puede ser configurado o usado pueden ser entendidas y probadas. Por ejemplo, si probamos un sitio web, puede no ser posible identificar y recrear

todas las configuraciones de hardware, sistema operativo, navegador, conexión, firewall y otros factores que el sitio web podría encontrar.

- **Diseñar la configuración del entorno de prueba e identificar cualquier infraestructura y herramientas necesarias.** Esto incluye herramientas de prueba y herramientas de soporte como hojas de cálculo, procesadores de texto, herramientas de planificación de proyectos y herramientas y equipos no informáticos, todo lo que necesitamos para llevar a cabo nuestro trabajo.

4. ¿Qué se hace en la implementación y ejecución de las pruebas?

Durante la implementación y ejecución de pruebas, tomamos las condiciones de prueba y las convertimos en casos de prueba y artefactos de prueba, y configuramos el entorno de prueba.

Esto significa que, habiendo elaborado un diseño de alto nivel para nuestras pruebas, ahora comenzamos a construirlas. Transformamos nuestras condiciones de prueba en casos de prueba y procedimientos, y otros artefactos de prueba, como scripts para automatización. También necesitamos configurar un entorno donde ejecutaremos las pruebas y construir nuestros datos de prueba. Configurar entornos y datos a menudo implica un tiempo y esfuerzo significativos, por lo que debes planificar y monitorear este trabajo cuidadosamente.

La **implementación** tienen las siguientes tareas principales, en aproximadamente el siguiente orden:

- **Desarrollar y priorizar nuestros casos de prueba**, utilizando técnicas, y crear datos de prueba para esas pruebas. También escribiremos instrucciones para llevar a cabo las pruebas (procedimientos de prueba). Para el examinador de manejo, esto podría significar cambiar la condición de prueba 'intersecciones' a 'tomar la ruta por Mayfield Road hasta la intersección con Summer Road y pedir al conductor que gire a la izquierda en Summer Road y luego a la derecha en Green Road, esperando que el conductor revise los espejos, señale y maniobre correctamente, mientras permanece consciente de otros usuarios de la carretera.' Es posible que necesitemos automatizar algunas pruebas utilizando arneses de prueba y scripts de prueba automatizados.

- **Crear suites de prueba a partir de los casos de prueba para una ejecución de pruebas eficiente.** Una suite de prueba es una colección lógica de casos de prueba que naturalmente funcionan juntos. Las suites de prueba a menudo comparten datos y un conjunto común de objetivos de alto nivel. También configuraremos un calendario de ejecución de pruebas.
- **Implementar y verificar el entorno.** Nos aseguramos de que el entorno de prueba se haya configurado correctamente, posiblemente incluso ejecutando pruebas específicas en él.

La **ejecución** tienen las siguientes tareas principales, en aproximadamente el siguiente orden:

- **Ejecutar las suites de prueba y los casos de prueba individuales, siguiendo nuestros procedimientos de prueba.** Podemos hacerlo manualmente o utilizando herramientas de ejecución de pruebas, de acuerdo con la secuencia planificada.
- **Registrar el resultado de la ejecución de pruebas y anotar las identidades y versiones del software bajo prueba, las herramientas de prueba y los artefactos de prueba.** Debemos saber exactamente qué pruebas utilizamos contra qué versión del software; debemos informar de los defectos contra versiones específicas; y el registro de pruebas que mantenemos proporciona una pista de auditoría.
- **Comparar los resultados reales** (lo que ocurrió cuando ejecutamos las pruebas) **con los resultados esperados** (lo que anticipamos que ocurriría).
- **Donde haya diferencias entre los resultados reales y los esperados, informar de las discrepancias como incidentes.** Los analizamos para reunir más detalles sobre el defecto, informando información adicional sobre el problema, identificando las causas del defecto y diferenciando entre problemas en el software y otros productos bajo prueba y cualquier defecto en los datos de prueba, en los documentos de prueba o errores en la forma en que ejecutamos la prueba. Querríamos registrar estos últimos para mejorar las pruebas en sí mismas.
- **Repetir las actividades de prueba como resultado de las acciones tomadas para cada discrepancia.** Necesitamos reejecutar las pruebas que previamente fallaron para confirmar una solución (pruebas de confirmación o re-pruebas). Ejecutamos pruebas y suites corregidas si hubo defectos en

nuestras pruebas. Probamos el software corregido nuevamente para asegurarnos de que el defecto fue efectivamente corregido (prueba de confirmación) y que los programadores no introdujeron defectos en áreas no modificadas del software y que la corrección de un defecto no descubrió otros defectos (pruebas de regresión).

5. ¿Qué se hace en la evaluación de los criterios de salida y reporte de pruebas?

Evaluar los criterios de salida es la actividad donde se evalúa la ejecución de las pruebas en función de los objetivos definidos. Esto debe hacerse para cada nivel de prueba, ya que para cada uno necesitamos saber si hemos realizado suficientes pruebas. Basándonos en nuestra evaluación de riesgos, habremos establecido criterios contra los cuales mediremos "suficiente".

Estos criterios varían para cada proyecto y se conocen como **criterios de salida**. Nos dicen si podemos declarar una actividad o nivel de prueba dado como completo. Podemos tener una combinación de **criterios de cobertura o finalización** (que nos indican los casos de prueba que deben incluirse, por ejemplo, "la prueba de manejo debe incluir una parada de emergencia" o "la prueba de software debe incluir una medición de respuesta"), **criterios de aceptación** (que nos indican cómo sabemos si el software ha pasado o fallado en general, por ejemplo, "solo aprobar al conductor si ha completado correctamente la parada de emergencia" o "solo aprobar el software para su liberación si cumple con la lista de requisitos de prioridad 1") y **criterios de salida del proceso** (que nos indican si hemos completado todas las tareas que necesitamos hacer, por ejemplo, "el examinador/probador no ha terminado hasta que haya escrito y archivado el informe de fin de prueba").

Los criterios de salida deben establecerse y evaluarse para cada nivel de prueba. Evaluar los criterios de salida tiene las siguientes tareas principales:

- **Verificar los registros de prueba contra los criterios de salida especificados en la planificación de pruebas:** Observamos qué evidencia tenemos de qué pruebas se han ejecutado y verificado, y qué defectos se han levantado, corregido, probado de confirmación o están pendientes.

- **Evaluar si se necesitan más pruebas o si los criterios de salida especificados deben cambiarse:** Es posible que necesitemos ejecutar más pruebas si no hemos ejecutado todas las pruebas que diseñamos, o si nos damos cuenta de que no hemos alcanzado la cobertura que esperábamos, o si los riesgos han aumentado para el proyecto. Podemos necesitar cambiar los criterios de salida para reducirlos, si los riesgos comerciales y del proyecto aumentan en importancia y los riesgos del producto o técnicos disminuyen en importancia. Ten en cuenta que esto no es fácil de hacer y debe ser acordado con las partes interesadas.
- **Escribir un informe de resumen de prueba para las partes interesadas:** No es suficiente que los evaluadores conozcan el resultado de la prueba. Todas las partes interesadas necesitan saber qué pruebas se han realizado y el resultado de las mismas, para tomar decisiones informadas sobre el software.

6. ¿Qué se hace en las actividades de cierre de prueba?

Durante las actividades de cierre de pruebas, recopilamos datos de las actividades de prueba completadas para consolidar la experiencia, incluyendo la verificación y archivo de los artefactos de prueba, y el análisis de hechos y cifras (**Métricas**). Es posible que necesitemos hacer esto cuando se entrega el software. También podemos cerrar las pruebas por otras razones, como cuando hemos reunido la información necesaria de las pruebas, cuando se cancela el proyecto, cuando se alcanza un hito particular o cuando se realiza una versión de mantenimiento o actualización.

Las actividades de cierre de pruebas incluyen las siguientes tareas principales:

- **Verificar qué entregables planeados realmente entregamos y asegurar que todos los informes de incidentes hayan sido resueltos a través de la reparación de defectos o la posposición.** Para los defectos pospuestos, es decir, aquellos que permanecen abiertos, podemos solicitar un cambio en una futura versión. Documentamos la aceptación o el rechazo del sistema de software.
- **Finalizar y archivar los artefactos de prueba, como scripts, el entorno de prueba y cualquier otra infraestructura de prueba, para su reutilización futura.** Es importante reutilizar todo lo que podamos de los artefactos de

prueba; inevitablemente realizaremos pruebas de mantenimiento, y ahorra tiempo y esfuerzo si nuestros artefactos de prueba pueden ser recuperados de una biblioteca de pruebas existentes. También nos permite comparar los resultados de las pruebas entre versiones de software.

- **Transferir los artefactos de prueba a la organización de mantenimiento que apoyará el software y realizará correcciones de errores o cambios de mantenimiento, para su uso en pruebas de confirmación y pruebas de regresión.** Este grupo puede ser diferente del grupo que construye y prueba el software; los evaluadores de mantenimiento son uno de los clientes de los evaluadores de desarrollo; ellos usarán la biblioteca de pruebas.
- **Evaluar cómo fue la prueba y analizar las lecciones aprendidas para futuras versiones y proyectos.** Esto puede incluir mejoras en el proceso para el ciclo de vida del desarrollo de software en su totalidad y también mejoras en los procesos de prueba. Si reflexionamos se podrían usar los resultados de las pruebas para establecer objetivos para mejorar las revisiones y pruebas con el objetivo de reducir el número de defectos en uso real.

Podríamos observar el número de incidentes que fueron problemas de prueba, con el objetivo de mejorar la forma en que diseñamos, ejecutamos y verificamos nuestras pruebas o la gestión de los entornos de prueba y datos. Esto nos ayuda a hacer que nuestras pruebas sean más maduras y rentables para la organización. Esto se documenta en un informe de resumen de pruebas o podría ser parte de un informe de evaluación general del proyecto.

Capítulo 18 - Roger S Pressman 7ma ed (técnicas de pruebas)

Libro:

■ Roger S Pressman - Ingeniería del software _ un enfoque práctico (7ma 2013)...

Preguntontas:

1. ¿Qué fundamentos existen para las pruebas del software?

Los fundamentos que existen para las pruebas es que en un sistema basado en computadoras o un producto debe diseñarse e implementarse teniendo en mente la **comprobabilidad**.

La **comprobabilidad** es saber con cuánta facilidad puede probarse. Para que se cumpla esto debe tener las siguientes características:

- **Operatividad.** “Mientras mejor funcione, más eficientemente puede probarse.” Si un sistema se diseña e implementa teniendo como objetivo la calidad, relativamente pocos errores bloquearán la ejecución de las pruebas, lo que permitirá avanzar en ellas sin interrupciones.
- **Observabilidad.** “Lo que ve es lo que prueba.” Las entradas proporcionadas como parte de las pruebas producen distintas salidas. Los estados del sistema y las variables son visibles o consultables durante la ejecución. La salida incorrecta se identifica con facilidad. Los errores internos se detectan y se reportan de manera automática. El código fuente es accesible.
- **Controlabilidad.** “Mientras mejor pueda controlar el software, más podrá automatizar y optimizar las pruebas.” Todas las salidas posibles pueden generarse a través de alguna combinación de entradas, y los formatos de entrada/salida (E/S) son consistentes y estructurados. Todo código es ejecutable a través de alguna combinación de entradas. El ingeniero de pruebas puede controlar directamente los estados del software, del hardware y las variables. Las pruebas pueden especificarse, automatizarse y reproducirse convenientemente.
- **Descomponibilidad.** “Al controlar el ámbito de las pruebas, es posible aislar más rápidamente los problemas y realizar pruebas nuevas y más inteligentes.” El sistema de software se construye a partir de módulos independientes que pueden probarse de manera independiente.

- **Simplicidad.** “Mientras haya menos que probar, más rápidamente se le puede probar.” El programa debe mostrar simplicidad funcional (por ejemplo, el conjunto característico es el mínimo necesario para satisfacer los requerimientos); simplicidad estructural (la arquitectura es modular para limitar la propagación de fallos) y simplicidad de código (se adopta un estándar de codificación para facilitar la inspección y el mantenimiento).
- **Estabilidad.** “Mientras menos cambios, menos perturbaciones para probar.” Los cambios al software son raros, se controlan cuando ocurren y no invalidan las pruebas existentes. El software se recupera bien de los fallos.
- **Comprensibilidad.** “Mientras más información se tenga, se probará con más inteligencia.” El diseño arquitectónico y las dependencias entre componentes internos, externos y compartidos son bien comprendidos. La documentación técnica es accesible al instante, está bien organizada, es específica, detallada y precisa. Los cambios al diseño son comunicados a los examinadores.

Una **buena prueba** tiene las siguientes características:

- Una buena prueba **tiene una alta probabilidad de encontrar un error**. Se debe hacer un imagen mental de cómo debería de fallar el software
- Una buena prueba **no es redundante**. Cada prueba debe tener un objetivo o propósito diferentes por los límites de los recursos.
- Una buena prueba **debe ser “la mejor de la camada”**. Si existen varias similares y los recursos son pocos, se debe elegir la que más probabilidad de encontrar un defecto.
- Una buena prueba **no debe ser demasiado simple o demasiado compleja**.

2. ¿Qué visiones existen de las pruebas?

La mayoría de productos sometidos a ingeniería pueden probarse en dos formas:

- Al conocer la función específica que se asignó a un producto para su realización, pueden llevarse a cabo pruebas que demuestren que cada función es completamente operativa mientras al mismo tiempo se buscan errores en cada función.
- Al conocer el funcionamiento interno de un producto, pueden realizarse pruebas para garantizar que las operaciones internas se realizan de acuerdo

con las especificaciones y que todos los componentes internos se revisaron de manera adecuada.

El primer enfoque se considera una **visión externa** y se denomina pruebas de **caja negra**.

El segundo se considera una **visión interna** y se denomina pruebas de **caja blanca**.

La **prueba de caja negra** se refiere a las pruebas que se llevan a cabo en la interfaz del software. Esta se encarga de examinar algunos aspectos fundamentales de un sistema con poca preocupación por la estructura lógica interna del software.

La **prueba de caja blanca** del software se basa en el examen cercano de los detalles de procedimiento. Las rutas lógicas a través del software y las colaboraciones entre componentes se ponen a prueba al revisar conjuntos específicos de condiciones y/o bucles.

3. ¿Qué son las pruebas de caja blanca y que tipos existen?

La **prueba de caja blanca** o **prueba de caja de vidrio** es una filosofía de diseño de casos de prueba que usa la estructura de control descrita como parte del diseño a nivel de componentes para derivar casos de prueba.

Ventajas de utilizar este enfoque:

- Garantizar que todas las rutas independientes dentro de un módulo se revisen al menos una vez.
- Revisar todas las decisiones lógicas en sus lados verdadero y falso.
- Ejecutar todos los bucles en sus fronteras y dentro de sus fronteras operativas.
- Revisar estructuras de datos internas para garantizar su validez.

Los tipos de pruebas de caja blanca que existen son:

- **Prueba de ruta o trayectoria básica.**

Esta técnica permite al diseñador de casos de prueba derivar una medida de complejidad lógica de un diseño de procedimiento y usar esta medida como guía para definir un conjunto básico de rutas de ejecución.

Para emplear esta técnica se debe primero realizar un **grafo de flujo** que es una representación abstracta del flujo de control. En este se muestra un flujo de control lógico que usa la notación de grafos. Cada nodo es uno o más enunciados de procedimiento. Las aristas representan el flujo del programa.

Para diseñar los casos de prueba se deben contemplar las **rutras independientes**, las cuales son las que introducen al menos un nuevo conjunto de enunciados de procesamiento o nueva condición en el programa, es decir, se debe recorrer al menos una arista que no se haya recorrido antes.

Para evitar sobreexigirse en buscar rutras independientes, se utiliza la **complejidad ciclomática** (Avalada por la teoría de grafos) que es una medición del software que proporciona una evaluación cuantitativa de la complejidad lógica de un programa. Aplicada en el contexto del grafo de flujo nos da un límite superior para la cantidad de rutras independientes. Las formas de calcularlo son las siguientes

1. El número de regiones del gráfico de flujo corresponde a la complejidad ciclomática.
2. La complejidad ciclomática $V(G)$ para un gráfico de flujo G se define como
$$V(G) = E - N + 2$$
$$E = \text{Cantidad aristas}$$
$$N = \text{Cantidad de nodos}$$
3. Cantidad de estructuras de control mas 1

Para cada ruta independiente se debe ejecutar al menos una vez para probar los caminos.

Una **matriz de grafo** es una matriz cuadrada cuyo tamaño (es decir, número de filas y columnas) es igual al número de nodos del gráfico de flujo. Cada fila y columna corresponde a un nodo identificado y las entradas de la matriz corresponden a conexiones (una arista) entre nodos.

- **Prueba de condición.** (Todas las técnicas de caja blanca son para probar estructuras de control)

En esta técnica se revisan las condiciones lógicas. Se clasifica en condiciones simples o compuestas.

Las **condiciones simples** son aquellas que es una variable booleana o expresión relacional ($a > b$) y capaz tengan el operador not.

Las **condiciones compuestas** son aquellas que se conforman con dos o mas condiciones simples, operadores booleanos (and o or).

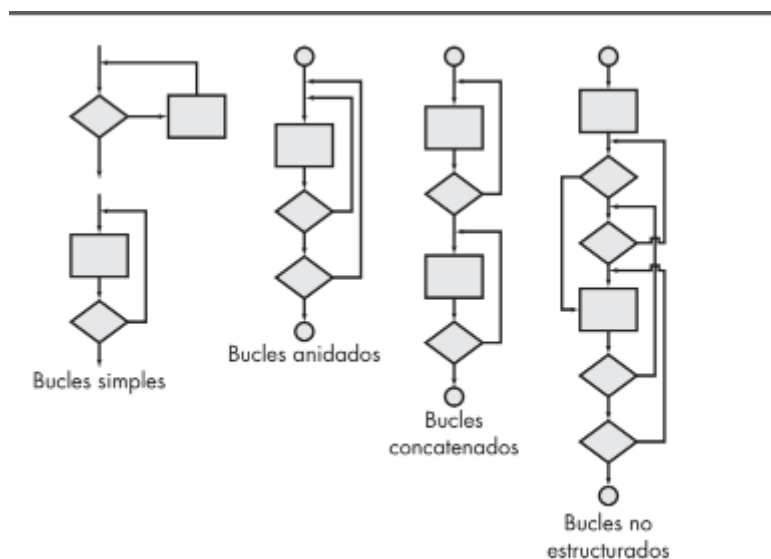
Se debe probar cada tipo de condición.

- **Prueba de flujo de datos.**

En este enfoque se seleccionan rutas del programa según las ubicaciones de las definiciones y con el uso de variables en el programa. Forma matemática de explicar el primero.

- **Prueba de bucle.**

Se enfocan exclusivamente en la validez de los constructores bucle. Los bucles se pueden clasificar en:



Para los **bucles simples** se pueden probar:

1. Saltar por completo el bucle.
2. Sólo una pasada a través del bucle.
3. Dos pasadas a través del bucle.
4. m pasadas a través del bucle, donde $m < n$.
5. $n - 1$, n , $n + 1$ pasadas a través del bucle.

Para los **bucles anidados** se pueden probar:

1. Comience con el bucle más interno. Establezca todos los otros bucles a valores mínimos.
2. Realice pruebas de bucle simple para el bucle más interno mientras mantiene los bucles exteriores en sus valores mínimos de parámetro de iteración (por ejemplo, contador de bucle). Agregue otras pruebas para valores fuera-de-rango o excluidos.
3. Trabaje hacia afuera y realice pruebas para el siguiente bucle, pero mantenga los otros bucles exteriores en valores mínimos y los otros bucles anidados en valores “típicos”.
4. Continúe hasta que todos los bucles se hayan probado.

Para los **bucles concatenados** se usan los dos anteriores.

Para los **bucles no estructurados** se debe rediseñar.

4. ¿Qué son las pruebas de caja negra y que tipos existen?

Las **pruebas de caja negra** o **pruebas de comportamiento** se enfocan en los requerimientos funcionales del software; es decir, las técnicas de prueba de caja negra le permiten derivar conjuntos de condiciones de entrada que revisarán por completo todos los requerimientos funcionales para un programa. Lo más probable es que este tipo de pruebas descubra diferentes errores que el de la caja blanca.

Errores que intenta encontrar este enfoque:

- Funciones incorrectas o faltantes
- Errores de interfaz
- Errores en las estructuras de datos o en el acceso a bases de datos externas
- Errores de comportamiento o rendimiento
- Errores de inicialización y terminación.

Los tipos de pruebas de caja negra que existe son:

- **Métodos de prueba basados en gráficos.**

Este método se basa en graficar los componentes de un sistema y sus relaciones para entenderlo.

- **Partición de equivalencia.**

Esta técnica se centra en dividir el **dominio de entrada** de un programa en clases (agrupación) de datos. Una **clase de equivalencia** representa un conjunto de estados válidos o inválidos para condiciones de entrada.

Las clases de equivalencia pueden definirse de acuerdo con los siguientes lineamientos:

1. Si una condición de entrada especifica un rango, se define una clase de equivalencia válida y dos inválidas.
2. Si una condición de entrada requiere un valor específico, se define una clase de equivalencia válida y dos inválidas.
3. Si una condición de entrada especifica un miembro de un conjunto, se define una clase de equivalencia válida y una inválida.
4. Si una condición de entrada es booleana, se define una clase válida y una inválida.

- **Análisis de valor de frontera.**

Este tipo de prueba conduce a una selección de casos de prueba que revisan los valores de frontera.

Los lineamientos son:

1. Si una condición de entrada especifica un rango acotado por valores a y b, los casos de prueba deben designarse con valores a y b, justo arriba y justo abajo de a y b.
2. Si una condición de entrada especifica un número de valores, deben desarrollarse casos de prueba que revisen los números mínimo y máximo. También se prueban los valores justo arriba y abajo, mínimo y máximo.
3. Aplicar lineamientos 1 y 2 a condiciones de salida. Por ejemplo, suponga que como salida de un programa de análisis de ingeniería se requiere una tabla de temperatura contra presión. Deben diseñarse casos de prueba para crear un reporte de salida que produzca el número máximo (y mínimo) permisible de entradas de tabla.
4. Si las estructuras de datos de programa internos tienen fronteras prescritas (por ejemplo, una tabla que tenga un límite definido de 100 entradas), asegúrese de diseñar un caso de prueba para revisar la estructura de datos en su frontera.

- **Prueba de arreglo ortogonal.**

Este tipo de pruebas se pueden aplicar en problemas en los que el dominio de entrada es relativamente pequeño pero demasiado grande para alojar la prueba exhaustiva.

5. ¿Qué pruebas existen para considerar arquitectura, entornos, etc?

Las pruebas que existen son:

- **Pruebas de interfaces gráficas de usuario.**

Las interfaces gráficas para usuario (GUI) presentan retos de prueba porque los componentes reutilizables son parte común en los entornos de desarrollo GUI, la creación de la interfaz para el usuario se ha vuelto menos consumidora de tiempo y más precisa. Pero, al mismo tiempo, la complejidad de las GUI ha crecido, lo que conduce a más dificultad en el diseño y ejecución de los casos de prueba. Es posible usar las gráficas de modelado de estado finito.

- **Prueba de arquitecturas cliente-servidor.**

La prueba del software cliente-servidor ocurre en tres niveles diferentes:

1. Las aplicaciones cliente individuales se prueban en un modo “desconectado”; no se considera la operación del servidor ni la red subyacente.
2. El software cliente y las aplicaciones servidor asociadas se prueban en concierto, pero las operaciones de red no se revisan de manera explícita.
3. Se prueba la arquitectura cliente-servidor completa, incluidos la operación de red y el rendimiento.

Por los niveles se pueden hacer las siguientes pruebas:

- **Pruebas de función de aplicación.** La funcionalidad de las aplicaciones cliente se prueba con los métodos de caja negra y blanca.
- **Pruebas de servidor.** Se prueban las funciones de coordinación y gestión de datos del servidor. También se considera el rendimiento del servidor (tiempo de respuesta global y cantidad de datos transmitidos).
- **Pruebas de base de datos.** Se prueba la precisión y la integridad de los datos almacenados por el servidor. Se examinan las transacciones colocadas por las aplicaciones cliente para asegurar que los datos se almacenen, actualicen y recuperen de manera adecuada. También se prueba la forma de archivar.

- **Pruebas de transacción.** Se crean una serie de pruebas para garantizar que cada clase de transacciones se procese de acuerdo con los requerimientos. Estas se enfocan en comprobar lo correcto del procesamiento y también en los conflictos de rendimiento (por ejemplo, tiempos de procesamiento de transacción y volumen de transacción).
- **Pruebas de comunicación de red.** Estas pruebas verifican que la comunicación entre los nodos de la red ocurre de manera correcta y que el mensaje que pasa, las transacciones y el tráfico de red relacionado ocurren sin errores. Como parte de estas pruebas, también pueden realizarse pruebas de seguridad de red.
- **Prueba para sistemas de tiempo real.** (Lo hago después)

6. ¿Qué son los patrones de prueba?

Los patrones pueden usarse para proponer soluciones a otras situaciones de ingeniería de software; en este caso, prueba del software. Los **patrones de prueba** describen problemas y soluciones de prueba comunes que pueden auxiliar en su tratamiento.

Los patrones de prueba no sólo proporcionan lineamientos útiles conforme comienzan las actividades de prueba; también proporcionan tres beneficios adicionales:

1. Proporcionan un vocabulario para quienes solucionan problemas. “Oiga, usted sabe, debemos usar un objeto nulo”.
2. Enfocan la atención en las fuerzas que hay detrás de un problema. Esto permite que los diseñadores [de caso de prueba] entiendan mejor cuándo y por qué se aplica una solución.
3. Alientan el pensamiento iterativo. Cada solución crea un nuevo contexto en el que pueden resolverse nuevos problemas.

Nombre del patrón: **PairTesting**

Resumen: Patrón orientado a proceso, **PairTesting** describe una técnica que es análoga a la programación por parejas (capítulo 3) en la que dos examinadores trabajan en conjunto para diseñar y ejecutar una serie de pruebas que pueden aplicarse a actividades de prueba de unidad, integración o validación.

Nombre del patrón: **SeparateTestInterface**

Resumen: Hay necesidad de probar cada clase en un sistema orientado a objetos, incluidas "clases internas" (es decir, clases que no exponen alguna interfaz afuera del componente que los usa). El patrón **SeparateTestInterface** describe cómo crear "una interfaz de prueba que puede usarse para describir pruebas específicas sobre clases que son visibles solamente de manera interna en un componente" [Lan01].

Nombre del patrón: **ScenarioTesting**

Resumen: Una vez realizadas las pruebas de unidad e integración, hay necesidad de determinar si el software se desempeñará en forma que satisfaga a los usuarios. El patrón **ScenarioTesting** describe una técnica para revisar el software desde el punto de vista del usuario. Un fallo en este nivel indica que el software fracasó para satisfacer un requisito visible del usuario [Kan01].

Preguntas del final

(Aclaración en este final nos pidieron que tengamos una carilla por pregunta solo que no lo dijeron hdps) Pueden cambiar las preguntas pero no el formato

1. **¿Qué es la validación y verificación de pruebas?**
2. **¿En qué momento termina el proceso de prueba?**
3. **¿Qué es una estrategia de pruebas (modelo v)?**
4. **¿Cuáles son las estrategias de pruebas genéricas (modelo v)?**
5. **¿Cuál es el propósito de una prueba de validación?**
6. **Explique las pruebas alfa y beta**
7. **¿Qué relación hay entre el plan de pruebas con el ciclo de vida del testing?**
8. **Explique las actividades de cada uno de los pasos del ciclo del testing**
9. **En las pruebas de caja negra que tipo de errores busca.**

Funciones incorrectas o faltantes

Errores de interfaz

Errores en las estructuras de datos o en el acceso a bases de datos externas

Errores de comportamiento o rendimiento

Errores de inicialización y terminación.

10. Explicar el método de partición equivalente

Este es el final del primer llamado oral.

- 1. ¿Por qué es importante la gestión de riesgos en la validación y verificación de software?**
- 2. ¿Cuáles son las categorías de riesgos?**
- 3. ¿Que es un caso de prueba? Explique dos técnicas de cada enfoque de casos de prueba (caja blanca y caja negra)**
- 4. ¿Qué es una inspección de software? Explique los pasos de las inspecciones de software**
- 5. ¿Cuáles son los pasos del ciclo de vida del testing? Explique que se hace en cada paso del ciclo de vida del testing**

- 1. Enumerar y explicar los distintos tipos de riesgos**
- 2. ¿Por qué es importante la gestión de riesgos en VyV?**
- 3. ¿Qué es la inspección de programas?**
- 4. Nombre y detalle los pasos de la inspección de programas.**

EXAMEN FINAL VALIDACIÓN Y VERIFICACIÓN DE SOFTWARE
diciembre 2022

Responda:

- a. ¿Qué es la validación y la verificación de software?
- b. ¿En que momento termina el proceso de prueba?

Responda:

- a. ¿Cuáles son los objetivos de las pruebas?
- b. Explique los enfoques del diseño de casos de prueba.
- c. Enuncie dos técnicas de cada uno de los enfoques de casos de prueba.

b) Responda:

- a. ¿Cuales son los pasos de ciclo de vida del testing?
- b. ¿Que relación hay entre el plan de pruebas y el ciclo de vida del testing?
- c. Explique que actividades se hacen en cada uno de los pasos del ciclo de vida del testing.

Preguntas que no entiendo pero le pregunte a la IA

1. ¿Que es un caso de prueba?

Los **casos de prueba** son un conjunto estructurado de entradas, acciones y condiciones diseñadas para verificar que el sistema se comporta según lo esperado.

Un caso de prueba típico incluye los siguientes elementos:

- **Identificador del caso de prueba:** Un identificador único para el caso de prueba.
- **Descripción:** Una breve descripción del objetivo del caso de prueba.
- **Condiciones previas:** El estado del sistema antes de ejecutar el caso de prueba.
- **Datos de entrada:** Los datos que se deben introducir en el sistema durante la prueba.
- **Pasos a seguir:** Una lista detallada de los pasos que se deben seguir para ejecutar el caso de prueba.
- **Resultados esperados:** La salida esperada del sistema si todo funciona correctamente.
- **Resultados reales:** La salida real del sistema después de ejecutar el caso de prueba.
- **Estado:** Una indicación de si el caso de prueba ha pasado o fallado, basándose en una comparación de los resultados esperados y reales.
- **Comentarios:** Notas adicionales, observaciones o recomendaciones sobre el caso de prueba.

Los casos de prueba se pueden crear para distintos tipos de pruebas, como pruebas funcionales, pruebas de rendimiento, pruebas de seguridad, entre otras.

2. ¿Qué relación hay entre el plan de pruebas y el ciclo de vida del testing?

El **plan de pruebas** es un documento que se crea durante la etapa de planificación de pruebas del ciclo de vida del testing. Este documento actúa como una hoja de ruta para todas las actividades de prueba.

La relación que posee con el ciclo de vida es que:

- **Guía para el Ciclo de Vida del Testing:** El plan de pruebas proporciona una guía clara y estructurada sobre cómo se llevarán a cabo las pruebas en cada etapa del ciclo de vida del testing.

- **Estandarización y Consistencia:** Ayuda a estandarizar el proceso de pruebas y asegura que todas las actividades de prueba se realicen de manera consistente y organizada.
- **Asignación de Recursos:** Facilita la asignación adecuada de recursos (humanos, técnicos, financieros) a lo largo de las distintas fases del ciclo de vida del testing.
- **Control y Seguimiento:** Permite el control y seguimiento de las actividades de prueba, asegurando que se cumplan los plazos y objetivos establecidos.
- **Evaluación de Progreso:** Facilita la evaluación del progreso de las pruebas y ayuda a identificar problemas o desviaciones del plan original.

En resumen, el plan de pruebas es un componente esencial dentro del ciclo de vida del testing, ya que define cómo se llevarán a cabo las pruebas y asegura que todas las actividades se alineen con los objetivos y requisitos del proyecto.

3. Explicame el Ciclo de vida del testing resumidamente

El **ciclo de vida del testing** se refiere a todas las etapas que se siguen para llevar a cabo las pruebas de software de manera sistemática y estructurada. Estas etapas típicamente incluyen:

- **Análisis de Requisitos:** Comprender y documentar lo que se debe probar.
- **Planificación de Pruebas:** Definir la estrategia, el alcance, los recursos, y el cronograma de las pruebas.
- **Diseño de Pruebas:** Crear casos de prueba detallados y preparar los datos de prueba necesarios.
- **Implementación de Pruebas:** Configurar el entorno de pruebas y preparar todo lo necesario para ejecutar los casos de prueba.
- **Ejecución de Pruebas:** Ejecutar los casos de prueba y registrar los resultados.
- **Evaluación de Resultados y Reporte de Defectos:** Comparar los resultados esperados con los resultados reales, y reportar cualquier defecto encontrado.
- **Cierre de Pruebas:** Evaluar la calidad del producto y las actividades de prueba, y preparar un informe final.

4. ¿Qué es una condición de prueba?

Una **condición de prueba** es una situación o conjunto de circunstancias que se deben verificar durante la ejecución de pruebas de software para determinar si el sistema o una parte del mismo se comporta según lo esperado. Las condiciones de prueba son específicas y se derivan de los requisitos del sistema, especificaciones de diseño, o cualquier otra fuente que defina el comportamiento esperado del sistema.

Características de una Condición de Prueba:

- **Específica:** Describe una situación particular que se debe probar.
- **Medible:** Permite verificar si el sistema pasa o falla la prueba basándose en criterios claros.
- **Relevante:** Relacionada directamente con los requisitos del sistema o sus componentes.

5. ¿Cuál es la diferencia entre diseñar e implementar?

La diferencia entre diseñar e implementar en el contexto de desarrollo de software (y otros campos) radica en los objetivos y actividades involucradas en cada etapa del proceso.

El objetivo principal del diseño es crear un plan o esquema detallado que guíe la construcción del sistema.

El objetivo principal de la implementación es construir y desarrollar el sistema de acuerdo con el diseño previamente definido.

6. ¿Qué significa evaluar la capacidad de las pruebas?

Evaluar la capacidad de las pruebas se refiere a la revisión y análisis de la efectividad y eficiencia del proceso de pruebas en un proyecto de desarrollo de software. Esto implica medir hasta qué punto las pruebas están cubriendo los requisitos del sistema, identificando defectos, y garantizando la calidad del producto final.

7. ¿Los métodos de caja blanca y caja negra son para diseñar casos de prueba?

Sí, los métodos de caja blanca y caja negra son enfoques utilizados para diseñar casos de prueba en el desarrollo de software. Cada método tiene su propio enfoque y se aplica en diferentes niveles del sistema para asegurar una cobertura de pruebas completa y efectiva.

8. ¿Qué son las inspecciones y qué diferencia existe con las revisiones?

Las inspecciones y revisiones son técnicas de aseguramiento de la calidad utilizadas en el desarrollo de software para identificar y corregir defectos en diversas etapas del ciclo de vida del desarrollo. Aunque ambos enfoques tienen el objetivo de mejorar la calidad del software, difieren en su metodología, formalidad y profundidad.

Una **inspección** es un proceso formal y sistemático de revisión de productos de trabajo (como código fuente, documentos de diseño, requisitos, etc.) con el objetivo de encontrar y corregir defectos. Las inspecciones son muy estructuradas y siguen un proceso definido.

Características:

- **Formalidad:** Altamente formal, con roles definidos y procedimientos específicos.
- **Roles:** Incluyen un moderador (facilitador), autor, lector, inspector y secretario (escriba).
- **Preparación:** Requiere una preparación previa por parte de los participantes, quienes revisan los productos de trabajo antes de la reunión de inspección.
- **Reunión:** Durante la reunión, el lector presenta el producto de trabajo mientras los inspectores buscan defectos.
- **Documentación:** Se documentan todos los defectos encontrados, y se elabora un informe detallado.
- **Seguimiento:** Se lleva a cabo un seguimiento para asegurar que todos los defectos encontrados sean corregidos.

Una **revisión** es un proceso menos formal que puede variar en cuanto a su rigor y metodología. El objetivo es identificar problemas y mejorar la calidad del producto de trabajo, pero con menos estructura y formalidad que las inspecciones.

Características:

- **Formalidad:** Puede variar de informal a semiestructurado, dependiendo del tipo de revisión.
- **Tipos:** Incluyen revisiones informales, revisiones por pares, revisiones técnicas, revisiones de gestión, etc.
- **Roles:** Pueden no estar tan estrictamente definidos como en las inspecciones. A menudo, solo participan el autor y uno o dos revisores.
- **Preparación:** Puede o no requerir una preparación previa extensa.
- **Reunión:** La estructura de la reunión puede ser flexible, y el objetivo principal es discutir el producto de trabajo y sugerir mejoras.
- **Documentación:** La documentación puede ser mínima o detallada, dependiendo de la formalidad de la revisión.
- **Seguimiento:** El seguimiento es menos riguroso que en las inspecciones, aunque se espera que se realicen las correcciones sugeridas.

9. ¿Las pruebas en sí validan y verifican?

Sí, las pruebas de software están diseñadas tanto para validar como para verificar el software, y estas dos actividades son componentes fundamentales del proceso de aseguramiento de la calidad. Aunque a menudo se mencionan juntas, la validación y la verificación tienen objetivos distintos en el contexto del testing de software.

Verificación

Objetivo: Asegurarse de que el software cumple con los requisitos y especificaciones establecidos en la fase de diseño.

Preguntas clave:

- ¿Estamos construyendo el producto correctamente?
- ¿El software se desarrolla de acuerdo con las especificaciones y estándares?

Actividades:

- Revisiones de Código: Verificación de que el código cumple con los estándares de calidad.
- Inspecciones: Evaluación detallada de los artefactos de software (código, documentos de diseño, etc.) para detectar defectos.
- Pruebas Estáticas: Análisis del código fuente sin ejecutarlo (análisis estático).
- Pruebas Unitarias: Verificación de que cada componente o módulo individual funciona correctamente.
- Pruebas de Integración: Verificación de que los módulos interactúan correctamente entre sí.

Enfoque: Interno, basado en el proceso y en la construcción del software.

Resultado: Confianza en que el software está construido de acuerdo con las especificaciones y que no tiene errores en su implementación o diseño.

Validación

Objetivo: Asegurarse de que el software cumple con las necesidades y expectativas del usuario final.

Preguntas clave:

- ¿Estamos construyendo el producto correcto?
- ¿El software cumple con los requisitos del usuario y proporciona el valor esperado?

Actividades:

- Pruebas de Sistema: Evaluación del sistema completo para verificar que cumple con los requisitos funcionales y no funcionales.
- Pruebas de Aceptación: Realizadas por los usuarios finales o un equipo de calidad para confirmar que el software cumple con sus necesidades.
- Pruebas de Usabilidad: Evaluación de la facilidad de uso y la experiencia del usuario con el software.
- Pruebas de Rendimiento: Validación de que el software cumple con los requisitos de rendimiento bajo diferentes condiciones de carga.

Enfoque: Externo, basado en los requisitos del usuario y la funcionalidad del sistema.

Resultado: Confianza en que el software satisface las necesidades del usuario final y está listo para su despliegue y uso en el entorno de producción.

Resumen de Diferencias

- Verificación:
 - Foco: Se enfoca en el proceso y en la construcción del software.
 - Pregunta: ¿Estamos construyendo el producto correctamente?
 - Métodos: Revisiones, inspecciones, pruebas unitarias e integración.
- Validación:
 - Foco: Se enfoca en el resultado final y en la satisfacción de los requisitos del usuario.
 - Pregunta: ¿Estamos construyendo el producto correcto?
 - Métodos: Pruebas de sistema, pruebas de aceptación, pruebas de usabilidad y rendimiento.

En resumen, verificación asegura que el software se está construyendo correctamente conforme a las especificaciones, mientras que validación asegura que el software correcto ha sido construido, cumpliendo las necesidades del usuario final. Ambos procesos son esenciales para garantizar la calidad y el éxito del proyecto de software.

10. Explique los enfoques del diseño de pruebas

El diseño de pruebas en el desarrollo de software puede abordarse desde diferentes enfoques, cada uno con sus propias técnicas y objetivos. Estos enfoques ayudan a asegurar que el software funcione según lo esperado y que cubra los requisitos tanto funcionales como no funcionales. Los principales enfoques del diseño de pruebas son:

- Enfoque de Caja Negra

También conocido como testing funcional, este enfoque se centra en las entradas y salidas del sistema sin tener en cuenta la estructura interna del código.

Objetivo: Validar que el software cumple con los requisitos funcionales especificados.

Técnicas:

- **Partición de Equivalencia:** Divide las entradas posibles en clases de equivalencia y selecciona casos de prueba representativos de cada clase.
- **Análisis de Valores Limítrofes:** Prueba los valores en los bordes de las clases de equivalencia, ya que los errores suelen ocurrir en los límites.
- **Tabla de Decisión:** Utiliza tablas para representar combinaciones complejas de condiciones y sus acciones correspondientes.
- **Pruebas Basadas en Casos de Uso:** Diseña casos de prueba basados en escenarios que describen cómo los usuarios interactuarán con el sistema.
- **Pruebas de Transición de Estados:** Evalúa cómo el sistema maneja las transiciones de un estado a otro, especialmente en sistemas con muchos estados posibles.

Ventajas:

- No requiere conocimiento del código fuente.
- Útil para verificar la funcionalidad completa desde la perspectiva del usuario final.

Limitaciones:

- Puede no cubrir todas las rutas internas del código.
- Depende en gran medida de la calidad de los requisitos.

- **Enfoque de Caja Blanca**

También conocido como testing estructural, este enfoque se centra en la estructura interna del código y el flujo de control del sistema.

Objetivo: Verificar que el software funcione correctamente a nivel interno y que todas las rutas posibles se ejecuten y se prueben.

Técnicas:

- **Cobertura de Sentencias:** Asegura que cada línea de código se ejecute al menos una vez.
- **Cobertura de Decisiones:** Asegura que cada decisión o condición (como if-else) se ejecute en ambas direcciones (verdadera y falsa).
- **Cobertura de Condiciones y Combinaciones de Condiciones:** Verifica que todas las condiciones en una decisión se prueben en todas sus combinaciones posibles.

- **Cobertura de Caminos:** Asegura que se prueben todas las rutas posibles a través del código.
- **Pruebas de Bucles:** Evalúa el comportamiento de los bucles, incluyendo la ejecución con diferentes números de iteraciones (incluyendo cero).

Ventajas:

- Permite una cobertura más completa del código.
- Ayuda a identificar errores en la lógica interna del sistema.

Limitaciones:

- Requiere un conocimiento profundo del código fuente.
- Puede ser costoso y complejo, especialmente en sistemas grandes y complejos.

Mas ejemplos

En una metodología basada en RUP (Rational Unified Process), las pruebas deben alinearse con las fases iterativas y la evolución del proyecto. A continuación, te propongo un esquema general de fechas y actividades para planificar el plan de pruebas, considerando revisiones técnicas y pruebas dinámicas:

1. Fase de Inicio

- **Objetivo:** Validar los requerimientos iniciales y preparar la estrategia de pruebas.
- **Fechas de pruebas:**
 - **Revisiones técnicas:** Última semana de la fase de inicio (revisión de requerimientos, validación de casos de uso preliminares).
 - **Actividades clave:** Revisión de documentos de requerimientos, validación inicial de riesgos de pruebas, y definición de estrategias y entornos.

2. Fase de Elaboración 1

- **Objetivo:** Validar modelos y arquitecturas base.
- **Fechas de pruebas:**
 - **Revisiones técnicas:** Mitad y final de la iteración.
 - **Pruebas dinámicas:** Fin de la iteración (pruebas sobre prototipos o módulos iniciales).
 - **Actividades clave:** Pruebas estáticas del modelo de diseño (diagramas UML) y prototipos.

3. Fase de Elaboración 2

- **Objetivo:** Refinar los módulos críticos y garantizar la estabilidad de los componentes principales.
 - **Fechas de pruebas:**
 - **Revisiones técnicas:** Mitad y final de la iteración.
 - **Pruebas dinámicas:** Fin de la iteración (pruebas de integración inicial de los módulos principales).
 - **Actividades clave:** Validación de componentes clave y pruebas de integración inicial.
-

4. Fase de Construcción 1

- **Objetivo:** Desarrollar y probar módulos funcionales iniciales.
- **Fechas de pruebas:**
 - **Revisiones técnicas:** Semana 2 y 4 (mitad y final de la iteración).
 - **Pruebas dinámicas:** Final de la iteración (pruebas unitarias e integración parcial).
 - **Actividades clave:** Pruebas funcionales de los módulos desarrollados en esta iteración.

Dijiste:

estoy cursando la materia verificación y validación de software y me surgió una duda con respecto a las revisiones e inspecciones, ¿cuál es la diferencia como sería una revisión y una inspección formal (detallando las principales diferencias) que se suele hacer en la práctica real?

ChatGPT dijo:

ChatGPT

Memoria actualizada

Las **revisiones** e **inspecciones** son técnicas utilizadas en la verificación y validación de software, pero tienen diferencias clave en términos de formalidad, objetivos y procesos. A continuación, te detallo sus características principales y cómo se aplican en la práctica:

Revisiones

- **Definición:** Actividades menos formales destinadas a identificar problemas en productos intermedios o finales del proceso de desarrollo de software.
 - **Participantes:** Generalmente involucran al equipo de trabajo (desarrolladores, analistas, testers), pero no requieren un moderador formal.
 - **Objetivo:** Detectar defectos, validar alineación con los requisitos y generar mejoras.
 - **Proceso:**
 - Se seleccionan los artefactos a revisar (código, documentación, requisitos, etc.).
 - Se realiza una reunión (opcional) para discutir puntos críticos.
 - Se entregan comentarios o informes con hallazgos.
 - **Formalidad:** Baja o media, no siempre documentada exhaustivamente.
 - **Ejemplo en la práctica:**
 - Una revisión de código entre pares (peer review) donde un compañero evalúa cambios antes de fusionarlos al repositorio.
-

Inspecciones

- **Definición:** Un proceso formal y estructurado para identificar defectos en un artefacto de software, con roles y procedimientos claramente definidos.
- **Participantes:**

- **Moderador:** Facilita la inspección y asegura que se sigan los pasos definidos.
 - **Autor:** Persona que creó el artefacto inspeccionado.
 - **Inspectores:** Evaluadores con experiencia técnica.
 - **Escriba:** Registra los hallazgos durante la inspección.
 - **Objetivo:** Detectar defectos específicos con un enfoque riguroso, buscando mejorar la calidad antes de pasar a etapas posteriores.
 - **Proceso:**
 - **Planeación:** El moderador organiza el proceso, selecciona artefactos y participantes.
 - **Preparación:** Los inspectores estudian el artefacto de manera individual antes de la reunión.
 - **Reunión formal:** Se discuten los defectos encontrados; no se resuelven problemas en este momento.
 - **Registro:** Se documentan todos los hallazgos y se generan reportes.
 - **Seguimiento:** El autor implementa las correcciones y el moderador verifica los resultados.
 - **Formalidad:** Alta, con documentos y guías que garantizan consistencia.
 - **Ejemplo en la práctica:**
 - Inspección formal de un documento de requisitos críticos para un sistema de misión crítica, siguiendo un checklist predefinido.
-

Principales Diferencias

Aspecto	Revisión	Inspección
Formalidad	Informal o semi-formal	Altamente formal
Estructura	Flexible	Rigurosamente definida
Participantes	Equipo técnico (sin roles fijos)	Roles definidos (moderador, escriba)
Objetivo	Mejora general y rápida	Identificar defectos específicos

Documentación	Opcional	Requiere registro detallado
Frecuencia	Más común en actividades diarias	Reservada para artefactos críticos

En la práctica real, las **revisiones** son más frecuentes, ya que son rápidas y menos costosas, mientras que las **inspecciones** se reservan para artefactos esenciales, como especificaciones de requisitos o diseños críticos, donde la calidad es prioritaria.

4o